

第 29 章: Class Coding Details (类编码细节)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本书对类机制 (class mechanics) 的第二轮深入讲解。第 27、28 章已完成 OOP 的"首轮冲刺", 本章则把 class 语句当作 Python 的"最后一个命名空间工具", 系统性地把类、方法、继承、命名空间与作用域规则整合成完整的故事——包括抽象超类、类数据属性、命名空间链接 (class/bases) 以及手动调用超类方法/构造器等核心概念, 为下一章的运算符重载 (operator overloading) 做铺垫。

29.1 开篇引言

英文原文

If you haven't quite grasped all of Python OOP yet, don't worry—now that we've taken a first pass, we're going to dig a bit deeper and study the concepts introduced earlier in further detail. In this and the following chapter, we'll take another look at class mechanics. Here, we'll study classes, methods, and inheritance, formalizing and expanding on some of the coding ideas introduced in Chapter 27 and demoed in Chapter

28. Because the class is our last namespace tool, we'll summarize Python's namespace and scope concepts as well. If you've been reading linearly, some of this chapter will be partly review and summary of topics introduced in the preceding chapter's case study, revisited here by language topics with self-contained examples that may help readers new to OOP. While you may be tempted to skip some material here, it includes extra details worth a browse and unveils more subtleties in Python's class model along the way. The next chapter continues this in-depth second pass over class mechanics by covering one specific aspect: operator overloading. First, though, let's fill in more of the Python OOP picture.

中文翻译

如果你还没有完全掌握 Python 的 OOP（面向对象编程），别担心——既然我们已经完成了第一轮浏览，接下来要挖得更深，更细致地研究前面介绍过的概念。在本章和下一章中，我们将再次审视类的机制。在这里，我们将学习类（class）、方法（method）和继承（inheritance），把第 27 章引入、第 28 章演示过的一些编码思想正式化并加以扩展。因为类是我们最后一个命名空间（namespace）工具，我们也会顺带总结一下 Python 的命名空间和作用域（scope）概念。如果你是按顺序阅读的，本章有一部分内容会是对前一章案例研究中引入主题的复习与总结——这里按语言主题重新组织，配以自成一体（self-contained）的示例，可能对新接触 OOP 的读者更有帮助。虽然你可能会

想跳过这里的某些内容，但它包含了一些值得翻阅的额外细节，并且沿途会揭示 Python 类模型中更多的精妙之处。下一章将继续这第二轮深入的类机制讲解，但只聚焦一个特定方面：运算符重载（operator overloading）。不过在此之前，让我们先把 Python OOP 这幅图画补充完整。

深度理解

·核心概念：本章是一次“二轮复习”——第一轮（第 27、28 章）解决“类怎么用”，这一轮解决“类到底是什么、名字是怎么被找到的”。作者明确告诉你：类是最后一个命名空间工具。此前你已见过函数命名空间（def）和模块命名空间（文件），类把这两者合二为一：它既是“局部作用域”（像函数），又是“属性容器”（像模块），还额外多出继承查找这层魔法。

·为什么这样设计：Mark Lutz 的教材一贯采用“先上手、后理论”的螺旋式结构。读者已经用类写过了（第 28 章的案例），所以本章可以放心地“拽掉魔术的幕布”，讨论查找机制等细节而不会吓跑初学者。

·生活例子：第 27、28 章像是“试驾”——先让你把车开起来；本章则是“打开引擎盖”——看看发动机、变速箱到底怎么协同工作，以及油路（命名空间）是怎么分布的。两轮缺一不可：只会开车不懂原理的人修不了车，只懂原理不会开车的人上不了路。

29.2 class 语句 (The class Statement)

英文原文

Although the Python class statement may seem similar to tools in other OOP languages on the surface, on closer inspection, it is quite different from what some programmers may be used to. For example, as in C++, the class statement is Python's main OOP tool, but unlike in C++, Python's class is not a declaration. Like a def, a class statement is an object builder and an implicit assignment—when run, it generates a class object and stores a reference to it in the name used in the header. Also like a def, a class statement is true executable code—your class doesn't exist until Python reaches and runs the class statement that defines it. This typically occurs while importing the module it is coded in, but not before.

中文翻译

虽然 Python 的 class 语句表面上可能与其他 OOP（面向对象编程）语言中的工具相似，但仔细观察就会发现，它与一些程序员习惯的东西大不相同。例如，与 C++ 一样，class 语句是 Python 主要的 OOP 工具，但与 C++ 不同的是，Python 的 class 不是声明（declaration）。像 def 一样，class 语句是一个对象构建器和隐式赋值——运行时，它会生成一个类对象（class object），并把对该对象的引用存储到头部（header）中使用的那个名字里。同样与 de

f 一样，class 语句是真正的可执行代码——在你的类被 Python 执行到并运行定义它的那条 class 语句之前，它并不存在。这通常发生在导入 (import) 它所在模块的时候，而不会更早。

深度理解

·核心概念：class 是"可执行的对象构建器"，不是"编译期声明"。C++/Java 里类在编译时就存在了，源码只是对编译器描述一个蓝图；Python 里类是运行时"长"出来的对象——class 语句被执行的那一刻，类对象才被创建并绑定到名字上。

·底层实现：解释器执行 class 语句时大致做三件事：① 执行类体内的所有语句（创建属性）；② 用这些属性构造一个新的类对象（其类型是 type，即元类）；③ 把类对象绑定到头部名字上。这三步全部发生在运行时 (runtime)，因此你可以把类放进 if 分支、函数内，甚至动态生成类——这是静态语言做不到的。

·为什么这样设计：把类当作普通对象 ("类也是对象，是 first-class citizen") 带来极大的灵活性：类可以存入列表、作为参数传递、被工厂函数生成。这一设计是 Python "everything is an object" 哲学的必然结果。

·生活例子：C++ 的类像"印刷厂的制版"——先有版，再印刷；Python 的类像"每次用烤箱现烤面包"——class 语句是"预热+烘焙"这个动作本身，执行一次出一块面包（一个类对象）。

·初学者误区：初学者常以为 class 语句是"模板定义"，把

它和实例混淆。记住：class 语句本身在被执行时只创建"类"这一个对象；创建实例要靠之后调用类（x = name(...)）。另外，类在 import 时才被创建，这意味着：如果模块从未被导入，类也从未存在。

29.3 一般语法与用法 (General Syntax and Usage)

英文原文

As we've seen, class is a compound statement with a body of statements typically indented under the header. In the header, superclasses are listed in parentheses after the class name, separated by commas. Listing more than one superclass leads to multiple inheritance, which we'll discuss more formally in Chapter 31 (in brief, the left-to-right order of superclasses in parentheses gives the search order). Here is the statement's general form and usage:

```
class name(superclass,...):  
    # Assign to name attr = value # Shared class data  
def method(self,...): # Methods  
    self.attr = value # Per-instance data  
x = name(...) # Make an instance  
x.method(...) # Call a method
```

Within the class statement, any assignments generate class attributes (both data items and callable functions known as methods); specially named methods implement built-in operations (e.g., a function named `init` is run at instance-object construction time if defined); and calling the class after its cla

ss has run makes instances of it.

中文翻译

正如我们已经看到的，class 是一个复合语句（compound statement），其语句体通常缩进在头部之下。在头部中，超类（superclass）列在类名后面的括号里，用逗号分隔。列出多个超类会导致多重继承（multiple inheritance），我们将在第 31 章更正式地讨论它（简而言之，括号中超类的从左到右顺序就是查找顺序）。下面是该语句的一般形式和用法：

```
class name(superclass,...): # 赋值给 name
    attr = value # 共享的类数据
    def method(self,...): # 方法
        self.attr = value # 每个实例各自的数据
x = name(...) # 创建一个实例
x.method(...) # 调用一个方法
```

在 class 语句内部，任何赋值都会生成类属性（class attribute）——包括数据项和称为方法（method）的可调用函数；特殊命名的方法实现内建操作（例如，若定义了名为 init 的函数，它会在实例对象构造时被运行）；而在 class 语句运行完之后调用这个类，就会创建它的实例（instance）。

代码分析

```
class name(superclass,...): # name— name
    attr = value # —
    def method(self,...): # —
        self.attr = value # —
x = name(...) # __init__
x.method(...) # x self
```

解释器如何处理这段代码：

1. 创建类对象：执行到 class 头部时，解释器建立一个新的“局部作用域”（即类体），依次执行其中每条语句。att

`r = value` 在类作用域里创建名字 `attr`; `def method` 创建函数对象并绑定到名字 `method`——两者最后都成为类对象的属性，存放在类的 `dict` 字典中。

2. 绑定类名：类体执行完毕后，解释器用 `type` 元类 (`metaclass`) 实例化出一个类对象，并把头部的名字 `name` 绑定到它。这一步与 `def` 把名字绑定到函数对象完全同构——这就是作者说“`class` 是隐式赋值”的原因。

3. 创建实例：`x = name(...)` 执行时，Python 调用类对象（类对象是“可调用的”），分配一个新实例，若类中定义了 `init` 则自动调用之，并把新实例作为 `self` 传入。

4. 方法调用：`x.method(...)` 的语义等价于 `name.method(x, ...)`——方法通过继承查找被定位到类对象上，`x` 被自动填入 `self` 参数。

设计思想：`class` 的语法刻意与 `def` 保持同构（“对象构建器 + 隐式赋值 + 可执行代码”三合一），降低了语言的心理负担；继承参数用括号跟在类名后，和函数调用的语法一致，暗示“超类列表”只是一个名字查找顺序表。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

29.4 示例：类属性 (Example: Class Attributes)

英文原文

As we've also seen, classes are mostly just namespaces—tools for defining names (i.e., attributes) that export data and logic to clients. Just as in a module file, the statements nested in a class statement body create its namespace and attributes. When Python reaches and runs a class statement, it runs all the statements nested in its body, from top to bottom. Assignments that happen during this process create names in the class's local scope, which become attributes in the associated class object. Because of this, classes resemble both modules and functions:

- Like functions, class statements are local scopes where names created by nested assignments live.
- Like modules, names assigned in a class statement become attributes in a class object.

The main distinction for classes is that their namespaces are also the basis of inheritance in Python: referenced attributes that are not found in a class or instance object may be fetched from other classes. B

because class is a compound statement, any sort of statement can be nested inside its body—print, assignments, if, def, and so on. All the statements inside the class statement run when the class statement itself runs (not when the class is later called to make an instance). Typically, assignment statements inside the class statement make data attributes and nested defs make method attributes. In general, though, any type of name assignment at the top level of a class statement creates a same-named attribute in the resulting class object. For example, assignments of simple nonfunction objects to class attributes produce data attributes shared by all instances. In the REPL of your choosing:

```
>>> class SharedData: attr = 16 # Generates a class
data attribute >>> x = SharedData() # Make two instances
>>> y = SharedData() >>> >>> x.attr, y.attr #
They inherit and share 'attr' (a.k.a. SharedData.attr) (16, 16)
```

Here, because the name `attr` is assigned at the top level of a class statement, it is attached to the class—which means it will be shared by all instances via the usual inheritance search from instance to class. We can change it by going through the class name, and we can refer to it through either instances or the class:

```
>>> SharedData.attr = 32 >>> >>> x.attr, y.attr, SharedData.attr (32, 32, 32)
```

Such class attributes can be used to manage information that spans all the instances—a counter of the number of instances generated, for example (an idea we'll expand on by example in Chapter 32). Now, watch what happens if we assign the name `attr` through an instance instead of the class:

```
>>> x.attr = 64 >>> >>> x.attr, y.attr, SharedData.attr (64, 32, 32)
```

Assignments to instance attributes create or change the names in the instance, not the shared class. More generally, inheritance searches occur only on attribute references, not on attribute assignments: assigning to an object's attribute always changes that object and no other (subject to the note ahead). For example, `y.attr` is still looked up in the class by inheritance, but the assignment to `x.attr` attaches a name to `x` itself and so replaces the version in the class. Readers who've done OOP before may recognize class attributes like `SharedData.attr` as similar to other languages' "static" data members—values that are stored in the class, independent of instances. In Python, it's nothing special: all class attributes are just names assigned in the class statement, whether they happen to reference functions or something else. When they are functions (a.k.a. methods), they simply receive an instance when called through one. Here's a more comprehensive example of this behavior that stores the same name in two places. Suppose we run the following class in a RE

PL:

```
>>> class MixedNames: # Define class data = 'text' #  
Assign class attr def init(self, value): # Assign method  
name self.data = value # Assign instance attr def disp  
lay(self): print(self.data, MixedNames.data) # Instance  
attr, class attr This class contains two defs, which assi  
gn class attributes to method functions. It also contain  
s a top-level = assignment statement; because this  
assignment assigns the name data inside the class, it  
lives in the class's local scope and becomes an attrib  
ute of the class object. Like all class attributes, this da  
ta is inherited and shared by all instances of the class  
that don't have data attributes of their own. When we  
make instances of this class, though, the name data  
is also attached to those instances by the assignment  
to self.data in the init method run automatically at in  
stance-construction time:
```

```
>>> x = MixedNames(1) # Make two instance objects  
>>> y = MixedNames(2) # Each has its own data  
>>>> x.display(); y.display() # self.data differs, Mix  
edNames.data is the same 1 text 2 text
```

The net result is that data lives in two places: in the instance objects (created by the self.data assignment in init) and in the class from which they inherit names (created by the data assignment in the class). The class's display method prints both versions by first qualifying the self instance and then the class. By using th

ese techniques to store attributes in different objects , we determine their scope of visibility. When attached to classes, names are shared. When attached to instances, names record per-instance data, not shared behavior or data. Although inheritance searches look up names for us, we can always get to an attribute anywhere in a tree by accessing the desired object directly. The object from which an attribute is requested focuses and limits search. In the preceding example, for instance, specifying `x.data` or `self.data` will return an instance name, which normally hides the same name in the class. However, `MixedNames.data` grabs the class's version of the name explicitly. The next section describes another common role for such through-the-class coding patterns and explains more about the way we deployed class-level fetches in the prior chapter.

中文翻译

正如我们所见，类在很大程度上只是命名空间——用来定义名字（即属性）的工具，向客户（client）导出数据和逻辑。就像在模块文件中一样，嵌套在 `class` 语句体中的语句会创建它的命名空间和属性。当 Python 执行到并运行一条 `class` 语句时，它会从上到下运行嵌套在类体中的所有语句。在此过程中发生的赋值会在类的局部作用域中创建名字，这些名字随后成为相关联的类对象中的属性。因此，类同时类似模块和函数：

- 像函数：`class` 语句是局部作用域，嵌套赋值创建的名字就生活在这里。
- 像模块：在 `class` 语句

中赋值的名字会成为类对象中的属性。类的主要独特之处在于：它们的命名空间同时也是 Python 中继承的基础——在类或实例对象中找不到的被引用属性，可以从其他类中获取。因为 class 是复合语句，任何种类的语句都可以嵌套在它的体内——print、赋值、if、def 等等。类语句内部的所有语句都在 class 语句本身运行时执行（而不是在之后调用类来创建实例时执行）。通常，类语句内部的赋值语句生成数据属性，嵌套的 def 生成方法属性。但一般来说，类语句顶层任何形式的名字赋值，都会在最终生成的类对象中创建一个同名的属性。例如，把简单的非函数对象赋给类属性，会产生所有实例共享的数据属性。在你喜欢的 REPL 中：

```
>>> class SharedData: attr = 16 # 生成一个类数据属性
>>> x = SharedData() # 创建两个实例
>>> y = SharedData()
>>> >>> x.attr, y.attr # 它们通过继承共享'attr'
(即 SharedData.attr) (16, 16)
```

这里，因为名字 attr 是在 class 语句顶层赋值的，所以它被挂到类上——这意味着它将通过从实例到类的常规继承查找被所有实例共享。我们可以通过类名来修改它，也可以通过实例或类来引用它：

```
>>> SharedData.attr = 32
>>> >>> x.attr, y.attr, SharedData.attr (32, 32, 32)
```

这样的类属性可以用来管理跨越所有实例的信息——例如已生成实例的计数器（这个想法我们将在第 32 章用示例展开）。现在，注意如果我们通过实例而不是类来赋值 attr，会发生什么：

```
>>> x.attr = 64
>>> >>> x.attr, y.attr, SharedData.attr (64, 32, 32)
```

对实例属性的赋值会在实例中创建或修改名字，而不是在共享的类中。更一般地说，继承查找只发生在属性引用 (reference) 上，而不发生在属性赋值 (assignment) 上：给一个对象的属性赋值，总是只改变该对象本身，不改变任何其

他对象（受后面的注意事项约束）。例如，`y.attr` 仍然通过继承在类中查找，但对 `x.attr` 的赋值把名字挂到了 `x` 自己身上，从而替换了类中的版本。有 OOP 经验的读者可能会把 `SharedData.attr` 这样的类属性类比为其他语言中的“静态”（static）数据成员——存储在类中、与实例无关的值。在 Python 中这没什么特别的：所有类属性都只是在 `class` 语句中赋值的名字，无论它们恰好引用函数还是别的东西。当它们是函数（即方法）时，通过实例调用时它们只是接收一个实例而已。下面是一个更全面的例子，演示了把同一个名字存储在两个地方的行为。假设我们在 REPL 中运行下面的类：

```
>>> class MixedNames: # 定义类
data = 'text' # 赋类属性
def init(self, value): # 赋方法名
self.data = value # 赋实例属性
def display(self): print(self.data, MixedNames.data) # 实例属性，类属性
这个类包含两个 def，它们把方法函数赋给类属性。它还包含一个顶层的 = 赋值语句；因为这个赋值把名字 data 赋在了 class 内部，它存在于类的局部作用域中，并成为类对象的属性。像所有类属性一样，这个 data 会被类中所有没有自己 data 属性的实例继承和共享。不过，当我们创建这个类的实例时，名字 data 也会通过 init 方法中（在实例构造时自动运行的）对 self.data 的赋值，被挂到这些实例上：
>>> x = MixedNames(1) # 创建两个实例对象
>>> y = MixedNames(2) # 每个都有各自的 data
>>> >>> x.display(); y.display()
# self.data 不同，MixedNames.data 相同
1 text 2 text
最终结果是 data 存在于两个地方：在实例对象中（由 init 中的 self.data 赋值创建）和在类中（由类中的 data 赋值创建，实例从类继承名字）。类的 display 方法先限定 self 实例、再限定类，从而打印出两个版本。通过用这些技巧把
```

属性存储在不同对象中，我们决定了它们的可见范围。挂到类上的名字是共享的；挂到实例上的名字记录的是每个实例各自的数据，而不是共享的行为或数据。虽然继承查找会替我们查找名字，但我们总能通过直接访问目标对象，到达树中任何位置的属性。请求属性的对象决定了查找的范围和边界。例如在前面的例子中，指定 `x.data` 或 `self.data` 会返回实例中的名字，它通常隐藏类中同名的那个。然而，`MixedNames.data` 显式地拿到了类的版本。下一节将描述这种"通过类访问"编码模式的另一个常见角色，并进一步解释我们在前一章中部署类级访问的方式。

代码分析

```
class SharedData:
    attr = 16                                #

x = SharedData()                            #
y = SharedData()
x.attr, y.attr                             # 'attr' SharedData.at...
# (16, 16)

SharedData.attr = 32                       # —""
x.attr, y.attr, SharedData.attr           # (32, 32, 32)

x.attr = 64                               # x
x.attr, y.attr, SharedData.attr           # (64, 32, 32)
```

```
class MixedNames:
    data = 'text'                          # —
    def __init__(self, value):             # —
        self.data = value                 # — data
    def display(self):
        print(self.data, MixedNames.data) #
```

解释器如何处理这段代码：

1. 类体执行阶段：执行 `class SharedData:` 时，解释器先建立一个空的作用域框架，把 `attr = 16` 放进去；类体结束，`attr: 16` 被写入新类对象的 `dict`。此时 `attr` 是类级名字。

2. 实例创建阶段：`x = SharedData()` 时，解释器分配一个全新的实例对象，其 `dict` 为空，但 `class` 指针指向 `SharedData`。

3. 继承查找协议（两步）：`x.attr` 求值时，解释器执行属性点号查找的两步协议：第一步在实例 `x.dict` 中查找 `attr`；找不到再走第二步，沿着 `class` 链（以及 `bases` 链）向上在类中查找。最初实例字典为空，于是命中类里的 `attr = 16`。

4. 赋值不进查找：`x.attr = 64` 是一个赋值，不是引用。赋值永远只写当前对象——解释器把 `attr: 64` 直接放进 `x.dict`。此后 `x.attr` 在第一步就命中（16 被 64 隐藏），而 `y.attr` 仍然两步走，落在类上（32）。这就是“同一个 `attr`，三个值”的底层解释：实例字典 vs 类字典是两个独立字典。

5. `MixedNames.data` vs `self.data`：`init` 里的 `self.data = value` 写入实例字典；类体里的 `data = 'text'` 写入类字典。`display` 里 `self.data` 走两步协议命中实例值，`MixedNames.data` 直接从类字典取——两种访问路径显式地展示了“同一名字、两个位置”。

设计思想：把“赋值永远只改本对象”定为铁律，配合“引用才触发继承查找”，使得 Python 的属性语义极度简单可预测：读可能“看”很远，写永远只写眼前。这也是为什么 Py

thon 不需要显式的 static 关键字——类的顶层赋值天然就是"静态成员"。

深度理解

·核心概念：类是命名空间，实例是另一个命名空间。类属性由类体顶层赋值生成（所有实例共享）；实例属性由 `self.attr = value` 生成（每实例独有）。继承查找是"引用时向上爬树"，赋值不爬树。

·底层实现：类对象和实例对象内部都是字典（dict）+ 链接（实例的 `class`、类的 `bases`）。"共享"不是复制——是查找时在类字典里"看见了同一个条目"。

·为什么这样设计：`class` 顶层赋值 = 静态成员，这消除了 C++/Java 里 `static` 关键字的特殊语法负担；同时坚持"赋值只改本对象"，让 Python 避免了很多动态语言里"不小心改了共享数据"的陷阱。

·实际问题：用类属性存计数器（实例计数）、共享配置、常量表等"横跨所有实例"的数据；用实例属性存每个对象自己的状态。

·初学者误区：① 误以为 `x.attr = 64` 会"更新共享值"——不会，它只是给 `x` 戴了一顶新帽子，藏住了类里的旧帽子；② 用可变对象做类属性（如 `class C: items = []`）时，`items.append(...)` 是修改而非赋值，会真实地改动共享列表，造成"一个实例改、所有实例都变"的经典 bug；③ 混淆"通过类赋值"（`SharedData.attr = 32`，合法改共享值）与"通过实例赋值"（`x.attr = 64`，只改实例）。

NOTE (赋值规则的例外)：给对象的属性赋值总是只改变该对象——除非该对象继承自一个用 `setattr` 运算符重载方法 (第 30 章讨论) 重新定义了属性赋值的类，或使用了属性 (properties)、描述符 (descriptors) 等高级属性管理工具 (第 32 章和第 38 章讨论)。本章大部分内容介绍的是常规情况，它对这个阶段和大多数 Python 代码来说已经足够。不过你以后会看到，Python 的类可是“装备丰富”——提供大量钩子 (hook) 让程序偏离常规——足以让这些简单规则变得五花八门。解读：记住这条“规则 + 例外”结构很重要。常规情况下“赋值只改本对象”，但 Python 提供了 `setattr`、`property`、`descriptor` 等拦截机制，可以改写这一行为——这是 Python“可塑性”的体现，也是后文第 38 章的重点。

29.5 方法 (Methods)

英文原文

Because you already know about functions, you also know about methods in classes. As you've learned, methods are just function objects created by `def` statements nested in a class statement's body. From an abstract perspective, methods provide behavior for instance objects to inherit. From a programming perspective, methods work in exactly the same way as simple functions, with one crucial exception: a method's first argument receives the instance object that is the implied subject of the method call. By way of review fro

m the last chapter, a method call made through an instance like this: `instance.method(args...)` is automatically translated into a call of method function in a class like this: `class.method(instance, args...)` where Python determines the class to use by locating the method name using the inheritance search procedure. In fact, both call forms are valid in Python: in the second, the class name narrows the method search, and the instance is provided explicitly, but the net result is the same for the same method. Besides the inheritance of method names, the special first argument is the only real magic behind method calls. In a class's method, the first argument is usually called `self` by convention (technically, only its position is significant, not its name). This argument provides methods with a hook back to the instance that is the subject of the call—because classes generate many instance objects, they use `self` to manage per-instance data. In Python, `self` is always explicit in your code: methods must always both list and use `self` to fetch or change attributes of the instance being processed by the current method call. This is by design—the presence of this name makes it obvious that you are using instance attribute names in your script, not names in the local or global scope.

中文翻译

因为你已经了解函数，所以你也了解类中的方法。正如你所学的，方法只是嵌套在 `class` 语句体中的 `def` 语句创建的函

数对象。从抽象角度看，方法为实例对象提供了可供继承的行为。从编程角度看，方法的工作方式与普通函数完全相同，只有一个关键例外：方法的第一个参数接收的是作为方法调用隐含主体的那个实例对象。回顾上一章的内容，通过实例进行的方法调用：`instance.method(args...)` 会被自动翻译成对类中方法函数的调用，就像这样：`class.method(instance, args...)` 其中 Python 通过继承查找过程 (inheritance search procedure) 定位方法名，从而确定要使用哪个类。事实上，两种调用形式在 Python 中都是合法的：第二种形式中，类名缩小了方法查找的范围，实例被显式提供，但对同一个方法而言，最终结果完全相同。除了方法名的继承之外，这个特殊的第一个参数是方法调用背后唯一的真正魔法。在类的方法中，第一个参数按惯例通常叫做 `self`（严格来说，只有它的位置有意义，名字本身无关紧要）。这个参数为方法提供了一条回到调用主体实例的“钩子”——因为类会生成许多实例对象，它们借助 `self` 来管理每个实例各自的数据。在 Python 中，`self` 在代码中总是显式的：方法必须同时列出 `self` 并使用 `self` 来获取或修改当前方法调用正在处理的实例的属性。这是有意为之——这个名字的存在让“脚本中你正在使用的是实例属性名”这一点一目了然，而不是局部或全局作用域中的名字。

深度理解

·核心概念：方法 = 函数对象 + 特殊的第一个参数 `self`。`instance.method(args)` 被语法糖翻译为 `class.method(instance, args)`。没有别的魔法——这是“方法”的全部秘密。

·底层实现：instance.method 求值时，解释器在类字典中找到函数对象，然后用绑定（binding）机制把它包装成“绑定方法”（bound method）对象——绑定方法内部记住了实例，调用时自动把实例塞进第一个参数。class.method(instance, args) 则是直接用未绑定（unbound）函数手动传参。绑定方法对象 x.method 和普通函数 NextClass.printer 是两类不同的对象，前者自带“记忆”。

·为什么这样设计：Python 选择显式 self（而不像 Java/C++ 用隐式 this），是为了让“读写实例属性”与“读写普通局部/全局变量”在词法上泾渭分明——self.attr 一看就知道是实例属性，attr 则按 LEGB 走。这消除了很多隐式 this 语言里的歧义。

·实际问题：多实例管理。类生成多个实例，方法必须知道“此刻在处理哪个实例”，self 就是那个指针——没有它，方法就无从区分 x 和 y 的数据。

·初学者误区：① 忘写 self 参数导致 TypeError ("takes 1 positional argument but 2 were given")；② 以为 self 是关键字——它只是约定俗成的名字，换成 this、me 都可以，但请永远用 self；③ 以为方法“属于”实例——其实方法存储在类里，实例只是借来用。

29.6 方法示例 (Method Example)

英文原文

To solidify these concepts, let's turn to an example. D

define the following class by running its code in a REPL:

```
>>> class NextClass: # Define class
def printer(self, text): # Define method
self.message = text # Change instance
print(self.message) # Access instance
```

The name `printer` references a normal function object; because it's assigned in the class statement's scope, it becomes a class-object attribute and is inherited by every instance made from the class—and earns the title `method`. Normally, because methods like `printer` are designed to process instances, we call them through instances:

```
>>> x = NextClass() # Make instance
>>> x.printer('instance call') # Call its method
instance call
>>> x.message # Instance changed
'instance call'
```

When we call the method by qualifying an instance like this, `printer` is first located by inheritance, and then its `self` argument is automatically assigned the instance object (`x`); the `text` argument gets the string passed at the call (`'instance call'`). Notice that because Python automatically passes the first argument to `self` for us, we can (and must) pass in just one argument. Inside `printer`, the name `self` is used to access or set per-instance data because it refers back to the instance currently being processed. As we've seen, though, methods may be called in one of two ways—through an instance or through the class itself. For example, we can also call `printer` by going through the class name, provided we pass an instance to the `self` argument explicitly:

```
>>> NextClass.printer(x, 'class call') # Direct cla
```

ss call class call >>> x.message # Instance changed a gain'class call' Calls routed through the instance and the class have the exact same effect—as long as we pass the same instance object ourselves in the class form. In fact, you get an error message if you try to call our method without any instance: >>> NextClass.printer('bad call') TypeError: NextClass.printer() missing 1 required positional argument:'text' Really, class methods are just functions assigned to class attributes, some of which happen to expect an instance that Python provides automatically only when methods are called through an instance. Moreover, calling a method through a class this way uses the same pattern we coded previously to fetch nonfunction class attributes. This same expression, class.attribute, works the same, whether the result is a callable object or not. It's a general tool.

中文翻译

为了巩固这些概念，我们来看一个例子。在 REPL 中运行下面的代码来定义这个类：>>> class NextClass: # 定义类
def printer(self, text): # 定义方法
self.message = text # 修改实例
print(self.message) # 访问实例名字
printer 引用一个普通的函数对象；因为它是在 class 语句的作用域中赋值的，它成为类对象的属性，并被从这个类创建的所有实例继承——从而赢得“方法”的头衔。通常，因为 printer 这样的方法是设计来处理实例的，我们通过实例来调用它们：
>>> x = NextClass() # 创建实例
>>> x.printer('instanc

e call') # 调用它的方法instance call >>> x.message # 实例被修改了'instance call'当我们这样通过实例限定来调用方法时，printer 首先通过继承被定位，然后它的 self 参数被自动赋值为该实例对象（x）；text 参数获得调用时传入的字符串（'instance call'）。注意，因为 Python 自动替我们把第一个参数传给 self，所以我们能（也必须）只传入一个参数。在 printer 内部，名字 self 被用来访问或设置每实例的数据，因为它指回当前正在被处理的实例。不过，正如我们所见，方法可以通过两种方式调用——通过实例，或通过类本身。例如，我们也可以通过类名来调用 printer，只要显式地把一个实例传给 self 参数：>>> NextClass.printer(x,'class call') # 直接通过类调用class call >>> x.message # 实例再次被修改'class call'通过实例和通过类路由的调用具有完全相同的效果——只要在类形式中我们自己传入同一个实例对象。事实上，如果你尝试不带任何实例调用我们的方法，会得到一条错误消息：>>> NextClass.printer('bad call') TypeError: NextClass.printer() missing 1 required positional argument:'text'其实，类方法只是赋给类属性的函数，其中一些恰好期待一个实例——而 Python 只在这类方法通过实例调用时才自动提供它。此外，这样通过类调用方法，与我们之前编写过的获取非函数类属性的模式是同一个模式。同一个表达式 class.attribute，无论结果是可调用对象与否，工作方式都一样。它是一个通用工具。

代码分析

```
>>> class NextClass:                                #
    def printer(self, text):                          #
        self.message = text                          #
        print(self.message)                          #
>>> x = NextClass()                                #
>>> x.printer('instance call')                       #
instance call
>>> NextClass.printer(x, 'class call')              #
class call
```

解释器如何处理这段代码：

1. 两种调用路径：x.printer('instance call') 走"绑定调用"：解释器先在类中定位 printer 函数对象，把 x 绑定进去生成绑定方法，调用时自动传 self=x、text='instance call'。NextClass.printer(x,'class call') 走"未绑定调用"：直接从类字典取普通函数对象，不生成绑定方法，x 和'class call' 都按普通位置参数传入——第一个落进 self，第二个落进 text。

2. 错误信息的意义：NextClass.printer('bad call') 中'bad call' 被当作 self 接收，text 没有值，所以报错 "missing 1 required positional argument:'text'"。这条错误信息是理解"方法 = 需要实例的函数"的最佳诊断窗口。

3. 数据落点：两种调用路径执行的是同一个函数体，self.message = text 都写入 x.dict，所以两次调用后 x.message 分别为'instance call' 和'class call'。

设计思想：class.attribute 是一个通用表达式——对非函数属性，它是"取数据"；对函数属性，它是"取函数（可能手动传实例）"。这种统一性让"显式调用超类方法"（如 Super.method(self)）成为可能，是后续"扩展继承方法"

模式的基石。

深度理解

·核心概念：两种调用等价——`x.method(a)` 与 `Class.method(x, a)` 是同一枚硬币的两面。差别只在“谁把实例塞进 `self`”：前者由绑定机制自动完成，后者由你亲手完成。

·底层实现：绑定方法是 CPython 中的 `method` 对象（由 `PyMethodNew` 创建），内部持有函数对象和实例（`self`、`func` 两个属性）。调用 `x.method(a)` 时，`call` 会展开成“函数对象(实例, a)”。这也解释了错误信息为何总是“缺了那个位置参数”。

·为什么这样设计：“自动传实例”只发生在通过实例取属性时，这保证了 `Class.method` 拿到的仍是普通函数——语言因此允许你把函数传来传去、手动调用，为装饰器、高阶函数等高级模式留出空间。

·实际问题：调试时若想调用某个方法的“原始版本”（绕过覆盖），`Class.method(instance, ...)` 是标准手段；很多框架的回调机制也依赖这一模式。

·初学者误区：① 以为方法必须有 `self` 才能工作——错，`self` 只是按位置接收实例的普通参数；② 以为 `x.printer` 和 `NextClass.printer` 是同一个对象——不是，前者是绑定方法、后者是函数，但它们调用起来殊途同归；③ 在 `print(x.printer)` 时会看到 `bound method NextClass.printer of <...object>`——这是调试时判断“绑定与否”的线索。

29.7 其他方法调用可能 (Other Method-Call Possibilities)

英文原文

This pattern of calling methods through a class is the general basis of extending—instead of completely replacing—inherited method behavior. It requires an explicit instance to be passed because all methods do by default. Technically, this is because methods called through instances are instance methods in the absence of any special code. In Chapter 32, we'll also study a less common option, static methods, that allows us to code methods that do not expect instance objects in their first arguments, even when called through an instance. Such methods can act like simple instanceless functions, with names that are local to the classes in which they are coded, and may be used to manage class data. A related concept we'll explore in the same chapter, class methods receive a class when called instead of an instance and can be used to manage per-class data, and are implied in metaclasses—yet another topic we'll reach later. These are all advanced, optional, and atypical extensions, though. Normally, an instance must always be passed to a method—whether automatically when it is called through an instance, or manually when you call through a class.

中文翻译

这种通过类调用方法的模式，是扩展（extending）——而不是完全替换——继承方法行为的一般基础。它要求显式传入一个实例，因为默认情况下所有方法都是如此。严格来说，这是因为在没有特殊代码的情况下，通过实例调用的方法就是实例方法（instance method）。在第 32 章中，我们还将研究一种不太常见的选项，静态方法（static method），它允许我们编写这样的方法：即使通过实例调用，它们的第一个参数也不期待实例对象。这类方法可以像简单的无实例函数一样工作，其名字局限于它们所编码的类中，并可用于管理类数据。在同一章中我们还会探讨一个相关概念，类方法（class method）：调用时接收的是一个类而不是实例，可用于管理每类的数据，而且在元类（metaclass）中也有体现——这是另一个我们以后才会触及的主题。不过，这些都是高级、可选、非典型的扩展。通常情况下，必须总是把实例传给方法——要么在通过实例调用时自动传递，要么在通过类调用时手动传递。

深度理解

·核心概念：默认的实例方法只是“函数 + 自动传实例”的合成品；静态方法（@staticmethod）和类方法（@classmethod）是刻意偏离这一默认的特殊变体。它们是高级工具，本章只需要知道它们存在。

·底层实现：@staticmethod 去掉绑定动作（调用时不自动传实例）；@classmethod 把绑定目标从“实例”换成“类”（cls 参数）。二者都要到第 32 章才正式登场，这里点到为止。

·为什么这样设计：方法并不需要"绑定"语义——绑定只是默认。Python 把"绑定与否"做成可配置项，让开发者可以按需选择方法的行为契约，这是 C++/Java 静态方法概念的更灵活版本。

·实际问题：静态方法常用于把与类相关的工具函数"收编"进类命名空间；类方法常用于替代构造函数（如 `cls.fromstring(...)` 这种"替代构造器"）或操作类级状态。

·初学者误区：初学者容易过早尝试 `@staticmethod`"让方法不用写 `self`"——但绝大多数场景下普通实例方法才是正确选择；使用 `@staticmethod` 通常只是把全局函数搬进类的命名空间。

29.8 继承 (Inheritance)

英文原文

Of course, the whole point of the namespace created by the class statement is to support name inheritance. This section expands on some of the mechanisms and roles of attribute inheritance in Python. As we've seen, in Python, inheritance happens when an object is qualified, and it involves searching an attribute definition tree—one or more namespaces. Every time you use an expression of the form `object.attr` where `object` is an instance or class object, Python searches the namespace tree from bottom to top, beginning with `object`, and looking for the first `attr` it can find. This al

so happens for references to self attributes in your methods. Because lower definitions in the tree override higher ones, inheritance forms the basis of specialization.

中文翻译

当然，class 语句创建的命名空间，其全部意义就在于支持名字继承（name inheritance）。本节将展开讨论 Python 中属性继承的一些机制和角色。正如我们所见，在 Python 中，继承发生在对象被限定（qualified）的时候，它涉及搜索一棵属性定义树——一个或多个命名空间。每当你使用 object.attr 这种形式的表达式（其中 object 是实例或类对象）时，Python 都会从下到上搜索这棵命名空间树，从 object 开始，寻找它能找到的第一个 attr。对方法中 self 属性的引用也是如此。因为树中较低的定义会覆盖较高的定义，继承构成了特化（specialization）的基础。

深度理解

- 核心概念：继承不是一个“复制行为”，而是一个运行时查找动作。object.attr 的求值 = 沿树从下往上找第一个 attr。这个查找动作只在“限定”（点号）发生时触发。
- 底层实现：查找路径是：实例 dict → 实例的 class 所指标的 dict → 该类的 bases 中各超类的 dict（多重继承时从左到右）→ 递归向上直到 object。找到即停，最先命中的胜出。
- 为什么这样设计：“查找胜出制”使子类重定义（override）天然就是“覆盖”——不需要 virtual 关键字，不需要声

明，只要同名即可。这极大降低了特化的语法成本。

·实际问题：这是框架设计的引擎——基类定义通用流程，子类只重定义"变化点"，系统通过查找自动把调用路由到子类版本。这是"开闭原则"（对扩展开放、对修改封闭）的运行时实现。

·初学者误区：① 以为子类"持有"父类的方法副本——不，子类字典里根本没有该方法，只是查找时"看得到"；② 以为继承只在类定义时发生——实际上每次点号求值都重新发生一遍，甚至可以通过 `x.attr = ...` 动态改变某个实例的"外观"；③ 把继承与包含（composition）混淆——继承是"是"（is-a）关系，不是"有"（has-a）。

29.9 属性树构建 (Attribute Tree Construction)

英文原文

Figure 29-1 summarizes the way namespace trees are constructed and populated with names. Generally:- Instance attributes are generated by assignments to self attributes in methods.- Class attributes are created by statements (assignments) nested in class statements.- Superclass links are made by listing classes in parentheses in a class statement header. Figure 29-1. Program code creates a tree of objects searched by attribute inheritance The net result is a tree of attribute namespaces that leads from an instance to the class

s it was generated from to all the superclasses listed in the class header. Python searches upward in this tree—from instances to superclasses, and left to right through multiple superclasses—each time you fetch an attribute name from an instance object.

中文翻译

图 29-1 总结了命名空间树是如何被构建并用名字填充的。一般来说：

- 实例属性：由方法中对 `self` 属性的赋值生成。
- 类属性：由嵌套在 `class` 语句中的语句（赋值）创建。
- 超类链接：通过在 `class` 语句头部括号中列出类来建立。

图 29-1. 程序代码创建一棵对象树，属性继承在树中搜索最终结果是一棵属性命名空间树，它从实例通向生成它的类，再通向 `class` 头部列出的所有超类。每当你从实例对象获取一个属性名时，Python 就沿这棵树向上搜索——从实例到超类，穿过多个超类时从左到右。

深度理解

·核心概念：整棵“属性树”由三样东西堆叠而成：实例字典、类字典、超类链接。树不是编译器画的，是代码执行时长出来的——`self.x = ...`长叶子（实例属性），类体赋值长树干（类属性），类头括号接枝（超类链接）。

·底层实现：实例的 `class` 指针是树的“根指针”；类的 `bases` 元组是“分叉点”。Python 的查找算法在 CPython 中由 `PyObjectGenericGetAttrWithDict` 等函数实现：先查实例，再沿 MRO（方法解析顺序，多重继承时的线性化顺序）向上。对单继承，MRO 就是从本类直通 `object` 的

一条链。

·为什么这样设计："向上查找"方向固定，保证下层的更特殊、上层的更通用，与"子类特化父类"的直觉完全一致。

·实际问题：排查属性来源时，大脑里要有这棵树：实例属性 (dict) > 类属性 > 超类属性。dir() 和 mro 可以帮你实际"看"这棵树。

·初学者误区：以为"继承 = 类之间物理上拷贝成员"。真相信这个说法的人，会在给类加属性后困惑"为什么实例忽然就多了个方法"。答案永远是：没有拷贝，只有查找。

29.10 继承细则 (Inheritance Fine Print)

英文原文

Technically speaking, the preceding description isn't complete because we can also create instance and class attributes by assigning them to objects outside of class statements. In Python, all attributes are always accessible by default, and privacy is an add-on (we'll talk about attribute privacy in Chapter 30 when we study setattr, in Chapter 31 when we meet X names, and again in Chapter 39 when we implement it with a class decorator). Even so, changes outside of a class are uncommon and error-prone: classes work best when they manage their instances. Also technically speaking, as hinted in Chapter 27, the full inheritance story grows more convoluted when advanced topics we hav

en't yet met are added to the mix. Metaclasses, diamond-pattern MROs, and descriptors, for example, may all play a role in some programs. Because of this, we'll begin formalizing the inheritance algorithm in Chapter 31 but won't finish it until Chapter 40. In the vast majority of Python code, though, inheritance is a simple way to redefine, and hence customize, behavior coded in classes—as the next section demos.

中文翻译

严格来说，前面的描述并不完整，因为我们也可以在 class 语句之外，通过给对象赋属性来创建实例属性和类属性。在 Python 中，所有属性默认总是可访问的，私有性（privacy）是一种附加功能（我们会在第 30 章学习 setattr 时、第 31 章遇到 X 名字时、以及第 39 章用类装饰器实现它时讨论属性私有性）。即便如此，在类外部进行修改并不常见且容易出错：类在“自己管理实例”时工作得最好。严格来说还有一点，正如第 27 章所暗示的，当我们将尚未遇到的高级主题加入混合后，完整的继承故事会变得更加错综复杂。例如元类（metaclass）、菱形模式 MRO（diamond-pattern MRO）和描述符（descriptor）都可能在某些程序中发挥作用。因此，我们将在第 31 章开始正式化继承算法，但要到第 40 章才能完成它。不过，在绝大多数 Python 代码中，继承只是重新定义——从而定制——类中编码行为的一种简单方式——下一节将演示这一点。

深度理解

·核心概念：本章描述的“简单继承模型”是一个有意简化的

版本。完整模型涉及元类、描述符、C3 线性化的 MRO 等高级机制，本书要到第 31、40 章才完整展开。先学简化版，再补高级版，是刻意安排的认知路径。

- 底层实现："菱形模式"指 `class D(B, C)` 且 B、C 都继承 A 的继承拓扑；此时 A 的方法可能被查找到两次，Python 用 C3 线性化算法保证每个类在 MRO 中只出现一次、且保持子类在前。这是第 31 章的重点。

- 为什么这样设计：Python 的"私有性"默认不存在，是因为"我们都是一致同意的成年人" ("We're all consenting adults") 哲学——不设墙，用下划线约定表达"请勿触碰"。这降低了语言复杂度，把约束权交还给程序员。

- 实际问题：类外补丁 (monkey-patching) 是存在的（比如给实例临时附加属性、给类挂方法），但它破坏封装、难以维护，团队项目里应避免。

- 初学者误区：以为"Python 有 `private/protected` 关键字"（没有）；以为下划线 `name` 开头的属性会被强制保护（不会，纯约定）；以为"覆盖"需要某种声明（不需要，同名即覆盖）。

29.11 特化继承方法 (Specializing Inherited Methods)

英文原文

The tree-searching model of inheritance just described turns out to be a great way to specialize systems. B

ecause inheritance finds names in subclasses before it checks superclasses, subclasses can replace default behavior by redefining their superclasses' attributes. In fact, you can build entire systems as hierarchies of classes, which you extend by adding new external subclasses rather than copying existing logic or changing it in place.

The idea of redefining inherited names leads to a variety of specialization techniques. For instance, subclasses may replace inherited attributes completely, provide attributes that a superclass expects to find, and extend superclass methods by calling back to the superclass from an overridden method. We've already seen some of these patterns in action; here's a self-contained example of extension at work:

```
>> class Super:
def method(self): print('in Super.method')
>> >> class Sub(Super):
def method(self): # Override method
print('starting Sub.method') # Add actions here
Super.method(self) # Run default action
print('ending Sub.method')
Direct superclass method calls are the crux of the matter here. The Sub class replaces Super's method function with its own specialized version, but within the replacement, Sub calls back to the version exported by Super to carry out the default behavior. In other words, Sub.method just exte
```

nds `Super.method`'s behavior rather than replacing it completely:

```
>> x = Super() # Make a Super instance
>> x.method() # Runs Super.method in Super.method
>> x = Sub() # Make a Sub instance
>> x.method() # Runs Sub.method, calls Super.method starting Sub.method in Super.method ending Sub.method
```

Perhaps the most common places that superclass-method calls show up are in constructors. The `init` method, like all attributes, is looked up by inheritance. This means that at construction time, Python locates and calls just one `init`, not one in every superclass.

If subclass constructors need to ensure that superclass construction-time logic runs too, they must call the superclass's `init` method explicitly. Calling it through the class name leverages the same general coding pattern we've been using in multiple roles:

```
>> class Super:
    def init(self, x): print('default code')
>> class Sub(Super):
    def init(self, x, y): Super.init(self, x) # Run superclass init
    print('custom code') # Do my extra init actions
>> I = Sub(1, 2)
default code
custom code
```

This is one of the few contexts in which your code is likely to call an operator-overloading method directly

. Naturally, you should call the superclass constructor this way only if you really want it to run—without the call, the subclass replaces it completely. For a more realistic illustration of this technique in action, see the Manager class example in the prior chapter's tutorial.

On a related note, readers with prior OOP experience may also be interested to know that redefining the constructor with differing argument lists in the same class means that only the last is used—later defs simply reassign the method name in Python. Starred arguments can be used in this role, but rarely are. We'll explore this phenomenon more fully in Chapter 31's coverage of polymorphism (short story: it's about interfaces, not call signatures).

中文翻译

前面描述的树查找式继承模型，被证明是特化（specialize）系统的绝佳方式。因为继承在检查超类之前先在子类中查找名字，子类可以通过重新定义超类的属性来替换默认行为。事实上，你可以把整个系统构建为类的层级结构，通过添加新的外部子类来扩展它，而不是复制现有逻辑或就地修改。重新定义继承名字的想法催生了多种特化技术。例如，子类可以完全替换（replace）继承的属性、提供（provide）超类期望找到的属性，以及通过在被覆盖的方法中回调超类来扩展（extend）超类方法。我们已经见过其中一些模式的实际运用；下面是一个自成一体（self-contained）的扩展示例：

```
>>> class Super: def method(self): print('in Super.method') >>> >>> class Sub(Super): def me
```

thod(self): # 覆盖方法print('starting Sub.method') # 在这里添加动作Super.method(self) # 运行默认动作print('ending Sub.method') 直接调用超类方法正是这里的要害。Sub 类用自己特化的版本替换了 Super 的 method 函数，但在替换内部，Sub 回调 Super 导出的版本来执行默认行为。换句话说，Sub.method 只是扩展了 Super.method 的行为，而不是完全替换它：>>> x = Super() # 创建 Super 实例>>> x.method() # 运行 Super.method in Super.method>>> x = Sub() # 创建 Sub 实例>>> x.method() # 运行 Sub.method, 内部调用 Super.method starting Sub.method in Super.method ending Sub.method

超类方法调用最常出现的地方也许是构造函数。init 方法像所有属性一样通过继承查找。这意味着在构造时，Python 定位并调用一个 init，而不是每个超类里的一个。如果子类构造函数需要确保超类的构造期逻辑也运行，就必须显式调用超类的 init 方法。通过类名调用它，利用了我们已在多种角色中使用过的同一个通用编码模式：>>> class Super: def init(self, x): print('default code')>>> >>> class Sub(Super): def init(self, x, y): Super.init(self, x) # 运行超类的 init print('custom code') # 做我自己额外的初始化动作>>> >>> l = Sub(1, 2) default code custom code 这是你的代码可能直接调用运算符重载方法的少数场景之一。当然，只有当确实想要超类构造函数运行时，才应该这样调用它——如果不调用，子类就完全替换了它。想看这个技巧更真实的演示，请参见前一章教程中的 Manager 类示例。顺带一提，有 OOP 经验的读者可能也有兴趣知道：在同一个类中用不同参数列表重新定义构造函数，意味着只有最后一个会被使用——在 Python 中，后面的 def 只

是重新赋值了方法名。星号参数 (starred arguments) 可以在这个角色中使用，但很少见。我们会在第 31 章讨论多态 (polymorphism) 时更充分地探讨这一现象 (简短版：它关乎的是接口，不是调用签名)。

代码分析

```
>>> class Super:
    def method(self):
        print('in Super.method')
>>> class Sub(Super):
    def method(self):
        print('starting Sub.method')
        Super.method(self)
        print('ending Sub.method')
>>> x = Sub()
>>> x.method()
starting Sub.method
in Super.method
ending Sub.method
```

解释器如何处理这段代码：

1. 方法分派 (dynamic dispatch) : `x.method()` 的查找沿 `Sub.dict` → 命中 `Sub.method`。因为查找“先见先得”，`Super.method` 被同名遮蔽——但遮蔽不等于删除，`Super` 字典里的函数对象依然完好。
2. 手动回拨：`Super.method(self)` 是“未绑定调用”——`Super.method` 从类字典取出普通函数，`self` (即 `x`) 手动传入。于是 `Sub.method` 先打印 `starting`，把控制权交给 `Super.method` (打印 `in Super.method`)，再打印 `ending`。调用链是显式的、可控的——你可以选择在开头、中间或结尾调用默认实现。

3. 构造函数的特殊性：Sub(1, 2) 时解释器只沿查找链调用一个 init——命中的是 Sub 里那个。Super.init 不会自动运行，必须由 Sub.init 显式调起。这就是为什么子类构造函数里几乎总能看到"先调 Super.init(self, ...)，再做自己的事"的固定句式。

4. 同名即覆盖的代价：类体里第二次 def init 只是把方法名重新绑定到新函数，旧函数被覆盖——所以 Python 没有 C++/Java 式的函数重载（同名不同签名）。多态不依赖调用签名，而依赖"对象是谁"（接口一致即可）。

设计思想：扩展而非替换，是"模板方法"（template method）设计模式的雏形：超类定义骨架与默认行为，子类用"覆盖 + 回调"注入定制。整个模式只依赖两条语言规则——"查找先见先得"和"类名访问可手动传实例"。

NOTE (super 提醒)：按照侧边栏"The super Alternative"的说法，Python 还有一个 super 内建函数，可以更通用地回调超类方法，但由于它的缺点和复杂性，我们把它的讲解推迟到第 32 章。不过，作为预告，前一节中的下面第一种写法也可以写成第二种——后者通过超出我们目前范围的深层魔法，本质上是自动化了 self 参数（注意它的小写）：
Super.method(self) # 显式，通用工具super().method() # 隐式，特例同样，本节中的构造函数调用也有同样的等价关系：Super.init(self, x) # 显式的基础写法super().init(x) # 隐式的替代写法不过，按照前述侧边栏的说法，super 在基本使用中有着众所周知的权衡，还有一个需要"全员配合"才能发挥最大效果的高级晦涩用例。由于这些问题，本书倾向于用显式名字调用超类，而不是 super。如果你是 Python 新手，建议遵循同样的策略，尤其是在你第一次过

OOP 的时候。现在先学会简单而通用的写法，以后再拿它和复杂而狭窄的写法做比较。

深度理解

- 核心概念：特化继承有三种姿势——替换（同名重定义，不留退路）、提供（实现超类期望的方法）、扩展（覆盖 + 显式回调超类版本）。三者都是"查找先见先得"规则的直接推论。
- 底层实现：`super()` 与 `Super.method(self)` 的差别在于查找起点：显式写法从 `Super` 开始找；`super()` 在单继承下也等价，但在多重继承下会沿 MRO 的"下一站"查找，行为更微妙（第 32 章详述）。
- 为什么这样设计：让"构造流程"也必须显式串联（`Super.init(self, ...)`），是 Python 对"初始化顺序"的诚实态度——语言不替你做决定，你写什么顺序就是什么顺序，避免隐式调用带来的意外。
- 实际问题：忘记在子类 `init` 中调用 `Super.init` 是最常见的 OOP bug 之一——症状是"父类字段全是初始值"。诊断方法：检查构造链。
- 初学者误区：① 以为"定义了子类 `init`，父类的就会自动先跑"——不会；② 以为 `def` 可以"重载"（同名多签名共存）——不会，后者覆盖前者；③ 过早使用 `super()`——本书建议初学者先掌握显式写法，理解机制后再引入 `super()`。

29.12 类接口技术 (Class Interface Techniques)

英文原文

Broadly speaking, the prior section's extension is only one way to interface with a superclass. The file listed in Example 29-1, `specialize.py`, defines multiple classes that illustrate a variety of common techniques:- Super — Defines a method function and a delegate that expects an action in a subclass- Inheritor — Doesn't provide any new names, so it gets everything defined in Super- Replacer — Overrides Super's method with a version of its own- Extender — Customizes Super's method by overriding and calling back to run the default- Provider — Implements the action method expected by Super's delegate method Study each of these subclasses to get a feel for the various ways they customize their common superclass.

中文翻译

广义地说，上一节的“扩展”只是与超类交互的一种方式。示例 29-1 中列出的文件 `specialize.py` 定义了多个类，演示了各种常见的技术：- Super ——定义了一个 method 函数和一个期望子类提供 action 的 delegate 方法- Inheritor ——不提供任何新名字，所以它获得 Super 中定义的一切- Replacer ——用自己的版本覆盖 Super 的 method- Extender ——通过覆盖并回调运行默认行为来定制 Super

的 method- Provider ——实现了 Super 的 delegate 方法所期望的 action 方法逐一研究这些子类，感受它们定制共同超类的各种方式。

代码分析

Example 29-1. specialize.py

```
class Super:
    def method(self):
        print('in Super.method') #
    def delegate(self):
        self.action()            #

class Inheritor(Super):          #
    pass

class Replacer(Super):          #
    def method(self):
        print('in Replacer.method')

class Extender(Super):          #
    def method(self):
        print('starting Extender.method')
        Super.method(self)      # super().method()
        print('ending Extender.method')

class Provider(Super):          #
    def action(self):
        print('in Provider.action')

if __name__ == '__main__':
    for klass in (Inheritor, Replacer, Extender):
        print('\n' + klass.__name__ + '...')
        klass().method()
    print('\nProvider...')
    x = Provider()
```

解释器如何处理这段代码：

1. 五个类，四种关系：Inheritor 的字典为空——查找 method 时沿 bases 直达 Super；Replacer 的字典有 method，Super 版本被遮蔽；Extender 覆盖后显式回调；Provider 覆盖 action（在 Super 里根本没有 action，Super 的 delegate 只是"期望"它存在）。

2. 类是一等对象：for klass in (Inheritor, Replacer, Extender) ——类被放进元组，klass() 直接调用类创建实例，klass.name 取类名。没有语法特殊待遇，因为类就是对象。

3. 延迟查找的威力：Super.delegate 中的 self.action() 在定义时无法解析——self 是谁要到运行时才知道。这就是"动态分派"（dynamic dispatch）：调用时沿 self 的类向上找 action，找到的是 Provider.action。

4. 运行输出：Inheritor().method() → in Super.method；Replacer().method() → in Replacer.method；Extender().method() → 三段式输出；Provider().delegate() → in Provider.action。跟踪每条输出，体会"查找发生在调用时刻"。

设计思想：这个文件是一张"类定制方式一览表"。四种定制方式（继承不动、完全替换、扩展叠加、填空实现）覆盖了框架开发中绝大多数与超类交互的场景，且全部只需要"同名定义"这一个语言机制。

深度理解

- 核心概念：类接口技术 = 子类与超类协作的四种姿势。前三种 (Inheritor/Replacer/Extender) 定制"已有方法"，第四种 (Provider) 实现"被期待的方法"——后者是抽象超类的雏形。
- 底层实现：pass 类体执行后字典里只有 module、doc 等内建条目，并不自动复制父类任何东西；"直到调用时才发现缺少 action"正是"异常在运行时才抛出的根源"。
- 为什么这样设计：把"需要子类做什么"交由运行时的动态查找去发现，框架作者无需知道子类细节——接口由"方法名 + 调用规约"隐式约定，而非显式接口声明。
- 实际问题：GUID 的 QWidget、Web 框架的 view 函数、XML 解析的回调等，都是"框架定义流程、你填实现"的实例。Provider 就是最小化的这种范式。
- 初学者误区：以为类必须先有"接口声明"才能被这样调用——Python 不需要；以为 pass 子类"什么都没有所以没用"——它恰好演示了"继承一切"是默认行为。

29.13 抽象超类 (Abstract Superclasses)

英文原文

Of the prior example's classes, Provider may be one of the most crucial to understand. When we call the delegate method through a Provider instance, two independent inheritance searches occur: 1. On the initial

x.delegate call, Python finds the delegate method in Super by searching the Provider instance and above. The instance x is passed into the method's self argument as usual. 2. Inside the Super.delegate method, self.action invokes a new, independent inheritance search of self and above. Because self references a Provider instance, the action method is located in the Provider subclass. This "filling in the blanks" sort of coding structure is typical of OOP frameworks. In a more realistic context, the method filled in this way might handle an event in a GUI, provide data to be rendered as part of a web page, process a tag's text in an XML file, and so on—your subclass provides specific actions, but the framework handles the rest of the overall job and runs your actions when needed. At least in terms of the delegate method, the superclass in this example is what is sometimes called an abstract superclass—a class that expects parts of its behavior to be provided by its subclasses. If an expected method is not defined in a subclass, Python raises an undefined name exception when the inheritance search fails. Class coders sometimes make such subclass requirements more obvious with assert statements or by raising the built-in NotImplementedError exception with raise statements. We'll study statements that may trigger exceptions in depth in the next part of this book; as a quick preview, here's the assert scheme in action:

```
>>> class Super: def delegate(self): self.action() def  
action(self): assert False,'action must be defined!' # Er  
ror if called
```

```
>>> X = Super() >>> >>> X.delegate() AssertionError  
or: action must be defined!
```

We'll study `assert` in Chapters 33 and 34; in short, if its first expression evaluates to false, it raises an exception with the provided error message. Here, the expression is always false so as to trigger an error message if a method is not redefined, and inheritance locates the stub version here. Alternatively, some classes simply raise a `NotImplementedError` exception directly in such method stubs to signal the mistake:

```
>>> class Super: def delegate(self): self.action() def  
action(self): raise NotImplementedError('action must  
be defined!') >>> X = Super() >>> >>> X.delegate()  
NotImplementedError: action must be defined!
```

For instances of subclasses, we still get the exception unless the subclass provides the expected method to replace the default in the superclass:

```
>>> class Sub(Super): pass >>> >>> X = Sub() >>>  
>>> X.delegate() NotImplementedError: action must  
be defined!
```

```
>>> >>> class Sub(Super): def action(self): print('ok  
ay') >>> >>> X = Sub() >>> >>> X.delegate() okay
```

For a somewhat more realistic example of this section's concepts in action, see the "Zoo animal hierarchy" exercise (Exercise 8) in "Test Your Knowledge: Part VI Exercises" and its solution in Appendix B. Such taxonomies are a traditional way to introduce OOP, but they're a bit removed from most developers' job descriptions (with apologies to any readers who happen to work at the zoo).

中文翻译

在前一个例子的类中，Provider 可能是最需要理解的关键之一。当我们通过 Provider 实例调用 `delegate` 方法时，发生了两趟相互独立的继承查找：1. 在最初的 `x.delegate` 调用上，Python 通过在 Provider 实例及其之上搜索，在 Super 中找到 `delegate` 方法。实例 `x` 照例被传入方法的 `self` 参数。2. 在 Super.`delegate` 方法内部，`self.action` 发起一趟新的、独立的对 `self` 及其之上的继承查找。因为 `self` 引用的是一个 Provider 实例，所以 `action` 方法在 Provider 子类中被找到。这种“填空”（filling in the blanks）式的编码结构在 OOP 框架中很典型。在更真实的场景中，以这种方式被填上的方法可能是：处理 GUI 中的一个事件、提供要渲染成网页一部分的数据、处理 XML 文件中某个标签的文本，等等——你的子类提供具体的动作，而框架负责其余的整体工作，并在需要时运行你的动作。至少就 `delegate` 方法而言，本例中的超类就是有时所说的抽象超类（abstract superclass）——一个期望其部分行为由子类提供的类。如果期望的方法在子类中没有定义，当继承查找失败时，Python 会抛出未定义名字的异常。类的编写者有时

会用 `assert` 语句或使用 `raise` 语句抛出内建的 `NotImplementedError`（未实现错误）异常，让这种子类要求更明显。我们将在本书下一部分深入研究可能触发异常的语句；这里先做个快速预览，看看 `assert` 方案的实际运用：

```
>>> class Super:
def delegate(self):
self.action()
def action(self):
assert False, 'action must be defined!'
# 若被调用则报错
>>> X = Super()
>>> X.delegate()
AssertionError: action must be defined!
```

我们将在第 33、34 章学习 `assert`；简而言之，如果它的第一个表达式求值为假，它就用提供的错误消息抛出一个异常。在这里，该表达式总是假，目的是在方法没有被重新定义时触发错误消息，而继承会定位到这里的桩（stub）版本。另外，有些类会直接在这样的方法桩中抛出 `NotImplementedError` 异常来示警：

```
>>> class Super:
def delegate(self):
self.action()
def action(self):
raise NotImplementedError('action must be defined!')
>>> X = Super()
>>> X.delegate()
NotImplementedError: action must be defined!
```

对于子类的实例，除非子类提供了期望的方法来替换超类中的默认版本，否则我们仍然会得到异常：

```
>>> class Sub(Super):
pass
>>> X = Sub()
>>> X.delegate()
NotImplementedError: action must be defined!
```

```
>>> class Sub(Super):
def action(self):
print('okay')
>>> X = Sub()
>>> X.delegate()
okay
```

要看本节概念更贴近实际的例子，请参见“Test Your Knowledge: Part VI Exercises”中的“Zoo animal hierarchy”练习题（练习 8）及其在附录 B 中的解答。这种分类学（taxonomy）是介绍 OOP 的传统方式，但它离大多数开发者的日常工作描述有点远（向碰巧在动物园工作的读者致歉）。

深度理解

·核心概念：抽象超类 = "只喊号子、不干活"的类：它定义流程（delegate）和被期待的方法名（action），但默认实现故意抛出异常，逼子类来填。两趟独立继承查找是理解它的钥匙——第一趟找 delegate（在本类），第二趟找 action（在 self 指向的子类）。

·底层实现：self.action() 中的 self 是运行时的动态绑定——x.delegate() 里 self 是 x（Provider 实例），所以第二趟查找从 Provider.dict 出发。assert False 生成一个总是求假的表达式，一旦执行到就抛 AssertionError；这是用"运行时失败"来强制"子类必须实现"的朴素方案。

·为什么这样设计：Python 直到 3.x 才引入正式的 abc 模块支持抽象基类。历史上，抽象超类就用 assert/NotImplementedError 这种"约定 + 运行时检查"实现——语言的灵活让"框架"可以在不新增语法的情况下实现类级约束。

·实际问题：GUI 事件处理、模板方法模式、插件系统..... 凡是"框架定流程、用户填钩子"的架构，几乎都是抽象超类。运行时抛错是最后一道防线，提醒你"忘了实现某方法"。

·初学者误区：① 以为抽象超类必须用 abc 模块——那是现代推荐做法，但非必须；② 以为异常应该掩盖在"返回空"里——尽早失败（fail fast）才是正确哲学，比起"悄悄什么都不做"，抛错对开发者友好得多；③ 忘掉"两趟查找"——一个方法内部的 self.xxx 会重新启动查找，这正

是多态的动态根源。

29.14 预览：用库工具实现抽象超类 (Preview: Abstract Superclasses with Library Tools)

英文原文

The preceding abstract superclasses (a.k.a. "abstract base classes"), which require methods to be filled in by subclasses, may also be implemented with special class syntax and a library module. This is coded with a keyword argument in a class header, along with special @ decorator syntax, both of which we'll study later in this book. While necessarily a preview in part, here is the special syntax equivalent of the preceding example:

```
>>> from abc import ABCMeta, abstractmethod
>>> >>> class Super(metaclass=ABCMeta):
def delegate(self): self.action() @abstractmethod
def action(self): pass
The net effect more rigidly prevents instance creation unless the method is defined lower in the class tree:
>>> X = Super()
TypeError: Can't instantiate abstract class Super without an implementation for abstract method'action'
>>> >>> class Sub(Super):
pass
>>> >>> X = Sub()
TypeError: Can't instantiate abstract class Sub without an implementation for abstract method'action'
>>> >>> class Sub(Super):
def action(self): print('okay')
>>> >>> X = Sub()
>>>
```

>>> X.delegate() okay Coded this way, a class with an abstract method cannot be instantiated (that is, we cannot create an instance by calling it) unless all of its abstract methods have been defined in subclasses. Although this requires more code and extra knowledge, the potential advantage of this approach is that errors for missing methods are issued when we attempt to make an instance of the class, not later when we try to call a missing method. This scheme may also be used to define an expected interface, automatically verified in client classes. Unfortunately, this scheme also relies on two advanced language tools we have not mastered yet—function decorators, introduced in Chapter 32 and covered in depth in Chapter 39, as well as metaclass declarations, mentioned in Chapter 32 and covered in Chapter 40—so we will postpone other facets of this option here. See Python's standard manuals for more on this, as well as precoded abstract superclasses Python provides.

中文翻译

前面的抽象超类（又叫“抽象基类”，abstract base class）——要求方法由子类填上——也可以用特殊的类语法和一个库模块来实现。这编码为 class 头部中的一个关键字参数（keyword argument），加上特殊的 @ 装饰器（decorator）语法，我们都会在本书后面学习。虽然这必然只是预览，但下面是前一例子的特殊语法等价实现：

```
>>> from abc import ABCMeta, abstractmethod
>>> >>> class
```

```
Super(metaclass=ABCMeta): def delegate(self): self.action() @abstractmethod def action(self): pass
```

净效果是更严格地阻止实例创建，除非该方法在类树更下方被定义：

```
>>> X = Super() TypeError: Can't instantiate abstract class Super without an implementation for abstract method'action'>>> >>> class Sub(Super): pass>>> >>> X = Sub() TypeError: Can't instantiate abstract class Sub without an implementation for abstract method'action'>>> >>> class Sub(Super): def action(self): print('okay')>>> >>> X = Sub()>>> >>> X.delegate() okay
```

这样编码后，一个含抽象方法的类不能被实例化（也就是说，我们不能通过调用它来创建实例），除非它的所有抽象方法都已在子类中被定义。虽然这需要更多代码和额外知识，但这种方式的潜在优势是：缺失方法的错误在我们尝试创建类的实例时就发出，而不是等到后来我们尝试调用缺失方法时才发出。这个方案也可用来定义期望的接口，并在客户端类中被自动验证。遗憾的是，这个方案还依赖于两个我们尚未掌握的高级语言工具——函数装饰器（function decorator，第 32 章引入、第 39 章深入讲解），以及元类声明（metaclass declaration，第 32 章提及、第 40 章深入讲解）——所以此处我们将推迟讨论这个选项的其他方面。更多内容以及 Python 提供的内置抽象超类，请参阅 Python 的标准手册。

代码分析

```

from abc import ABCMeta, abstractmethod      # abc

class Super(metaclass=ABCMeta):               # ABCMeta
    def delegate(self):                       #
        self.action()
    @abstractmethod                           # action
    def action(self):
        pass                                  #

```

解释器如何处理这段代码：

1. 元类接管类创建：metaclass=ABCMeta 告诉解释器：创建 Super 类对象时，不要用默认的类型元类，而用 ABCMeta。类对象创建后，身上会长出一个 abstractmethods 集合。@abstractmethod 把 action 登记进这个集合，并让它"带病标记"进入类字典。
 2. 实例化拦截：调用 Super() 或 Sub() 时，ABCMeta.call 检查新建类（及沿 MRO 汇总）的抽象方法集合是否为空。只要还有未实现的抽象方法，就抛出 TypeError，根本到不了 init。这就是"创建时即报错"（而不是"调用缺失方法时才报错"）的底层原理。
 3. 继承链接力检查：class Sub(Super): pass 继承了 action（抽象实现），所以 Sub 的抽象集合仍非空，Sub() 一样被拦下。直到 Sub 自己定义了 action，覆盖了抽象版本，抽象集合清零，实例化才放行。
 4. 运行差异：对比 29.13 的 assert/NotImplementedError 方案——那是"调用到才炸"，abc 方案是"实例化就炸"。后者把错误从运行时提前到了更早的构造阶段。
- 设计思想：abc 方案把"抽象约束"从约定提升为语言的强

制力——用类型系统（元类）层面的检查，保证“不完整类永远无法诞生”。这是“接口定义 + 自动验证”在现代 Python 中的标准做法，也是 DDD/架构校验等场景的基石。现代 Python 中还可以直接继承 ABC 而不是手动写 `metaclass=ABCMeta`，效果相同。

深度理解

·核心概念：抽象基类（ABC）是抽象超类的“正规军”版本——用 `@abstractmethod` 声明“必须实现的方法”，用元类拦截不完整的实例化。本书此处只做预览，详细机制留到第 39、40 章。

·底层实现：`abc` 模块 = 元类（拦截实例化）+ 装饰器（登记抽象方法）+ 内建容器来存储“谁还没被实现”。核心是“抽象集合为空才允许实例化”这条检查规则。

·为什么这样设计：“尽早失败”原则的极致化——与其让错误在运行时深处冒泡，不如在对象根本构不出来的最早期就拒绝。这也是对“契约式设计”（design by contract）的支持。

·实际问题：设计大型库、插件体系、类型标注（`typing.Protocol` 是其运行时的结构化变体）时，用 ABC 定义“这个接口保证提供这些方法”，让 IDE、类型检查器（`mypy/pyright`）和运行时三方共同守住契约。

·初学者误区：① 以为“加了 `@abstractmethod` 就自动是抽象类”——还必须让类的元类识别它；② 以为装饰器是“只能用的黑魔法”——第 39 章会揭开它的面纱：装饰器本质就是一个“接收函数、返回函数”的函数；③ 以为 ABC

和 Java 接口完全等价——ABC 允许非抽象方法、默认实现和 subclasshook，比接口更灵活。

29.15 命名空间：总结 (Namespaces: The Conclusion)

英文原文

Now that we've examined class and instance objects, the Python namespace story is complete. For reference, this section summarizes all the rules used to resolve names and extends them to classes. The first things you need to remember are that qualified and unqualified names are treated differently, and that some scopes serve to initialize object namespaces:- Unqualified names (e.g., X) deal with scopes.- Qualified attribute names (e.g., object.X) use object namespaces.- Some scopes initialize object namespaces (for modules and classes). These concepts sometimes interact—in object.X, for example, object is first looked up per scopes, and then X is looked up in the located object. Since scopes and namespaces are essential to understanding Python code, let's flesh out the rules in more detail.

中文翻译

既然我们已经考察了类和实例对象，Python 的命名空间故事就完整了。为便于参考，本节总结了用于解析名字的所有

规则，并把它们扩展到类。首先要记住的是：限定名 (qualified) 与非限定名 (unqualified) 的处理方式不同，而且有些作用域的作用是初始化对象的命名空间：- 非限定名 (如 X) 走作用域 (scope) 规则。- 限定属性名 (如 object.X) 使用对象命名空间 (object namespace)。- 有些作用域会初始化对象的命名空间 (对于模块和类)。这些概念有时会相互作用——例如在 object.X 中，object 首先按作用域规则查找，然后在定位到的对象中查找 X。由于作用域和命名空间对理解 Python 代码至关重要，让我们更详细地展开这些规则。

深度理解

·核心概念：Python 里"一个名字"有两种完全不同的解析机制——裸名字 (X) 走 LEGB 词法作用域链；限定名字 (object.X) 走对象命名空间 + 继承查找。二者不可混淆，这是全章最重要的分野。

·底层实现：裸名字在编译期就被确定归属 (编译器把函数内的 X 编译成 LOADFAST/LOADGLOBAL 等专用指令)；限定名字编译成 LOADATTR，运行时动态查找。所以裸名字是"静态决定的"，限定名是"动态决定的"——这解释了为什么裸名字更快、更受限。

·为什么这样设计：作用域是"编译期可见的文本结构"，命名空间是"运行时对象携带的数据"。把两者分开，Python 既有了词法闭包的清晰性，又有了对象属性的动态性——self.X 可以任意增删，X 却必须遵循作用域规则。

·实际问题：诊断 NameError vs AttributeError：X 报 NameError 说明作用域链里没有它；obj.X 报 AttributeEr

ror 说明对象链里没有它。错误类型本身就在告诉你"走错了通道"。

·初学者误区：在方法里写 `data = 'text'`（类体里那个）想读类属性——`data` 是裸名字，走的是作用域链，永远看不见类的属性；必须写 `self.data` 或 `ClassName.data`。这是 LEGB 里"类不参与"这一条最典型的踩坑现场。

29.16 简单名字：除非赋值否则全局 (Simple Names: Global Unless Assigned)

英文原文

As we've seen, unqualified simple names follow the LEGB lexical scoping rule outlined when we explored functions in Chapter 17:- Assignment (`X = value`) — Makes names local by default: creates or changes the name `X` in the current local scope, unless declared global or nonlocal in that scope. These declarations work in both `def` for functions and `class` for classes.- Reference (`X`) — Looks for the name `X` in the current local scope (`L`), then any and all enclosing functions from inner to outer (`E`), then the current global-scope module (`G`), then the built-ins module (`B`)—per the LEGB rule. Notably absent here, enclosing classes are not searched; class names are referenced as object attributes instead. Also per Chapter 17, some special-case constructs localize names further (e.g., variables in som

e comprehensions and some try statement clauses), and nested class scopes currently have some peculiarities that reflect longstanding bug reports and are too obscure to merit coverage here. The vast majority of names, however, follow the LEGB rule. New here, the class statement allows global and nonlocal to modify assignment rules the same as def, though we have to nest it to see how the latter of these come online: >>> gvar = 111 >>> class C: global gvar # Change name gvar in enclosing module gvar = 222 # Else it would be class attribute C.gvar >>> gvar 222 >>> >>> def outer(): nvar = 111 class C: nonlocal nvar # Change name nvar in enclosing function nvar = 222 # Else it would be class attribute C.nvar print(nvar) >>> >>> outer() 222 Though rare, namespace declarations in class map assignments to outer scopes, instead of making class attributes—the same way they prevent assignments from making local variables in functions. There's more on how nested classes interact with scopes in default cases later in this section.

中文翻译

正如我们所见，非限定的简单名字遵循第 17 章讨论函数时概述的 LEGB 词法作用域规则：- 赋值 ($X = \text{value}$) ——默认使名字成为局部 (local) 的：除非在该作用域中声明了 global 或 nonlocal，否则会在当前局部作用域中创建或修改名字 X。这些声明在 def (函数) 和 class (类) 中都有效。- 引用 (X) ——按 LEGB 规则，先在当前局部作用

域 (L) 中查找名字 X, 然后在从内到外的所有包围函数 (E) 中查找, 再在当前全局作用域模块 (G) 中查找, 最后在内建模块 (B) 中查找。这里值得特别注意的缺席者: 包围的类不会被搜索; 类名改为通过对象属性来引用。同样按照第 17 章的说法, 一些特殊构造会进一步局部化名字 (例如某些推导式 (comprehension) 和一些 try 语句子句中的变量), 而且嵌套类作用域目前存在一些反映长期 bug 报告的怪癖, 太过晦涩, 不值得在此展开。不过, 绝大多数名字都遵循 LEGB 规则。本章的新内容是: class 语句允许 global 和 nonlocal 像 def 一样修改赋值规则, 不过我们必须嵌套它才能看到后者 (nonlocal) 如何上线: >>> gvar = 111 >>> class C: global gvar # 修改外层模块中的名字 gvar gvar = 222 # 否则它会是类属性 C.gvar >>> gvar = 222 >>> >>> def outer(): nvar = 111 class C: nonlocal nvar # 修改外层函数中的名字 nvar nvar = 222 # 否则它会是类属性 C.nvar print(nvar) >>> >>> outer() 222 尽管罕见, class 中的命名空间声明会把赋值映射到外层作用域, 而不是制造类属性——这与它们阻止赋值在函数中制造局部变量的方式相同。关于嵌套类在默认情况下如何与作用域交互, 本节后面还有更多内容。

深度理解

- 核心概念: 类的顶层作用域 = LEGB 里的 L (局部), 但它不是任何代码的包围作用域 (E)。方法内的裸名字查找会“跳过”类, 直接去看外层函数/模块/内建。global/nonlocal 在类体内与在函数体内一样有效。
- 底层实现: 编译器在编译类体时, 把它当作一个特殊的代码对象处理: 类体内的裸名字赋值默认进入“类局部” (最

终倒入类字典)；而方法函数的代码对象是独立的，它的闭包链 (closure) 只会包含外层函数作用域——类不在其中。这就是"方法里看不到类体局部名"的字节码级根源。

·为什么这样设计：方法频繁使用 self 访问实例，若类又参与 LEGB，名字解析就会混乱（"这个 X 到底指类属性还是全局？"）。作者用"类被排除在 E 之外"这一刀，换来了规则的绝对清晰。

·实际问题：global/nonlocal 在类体里极少用到，但"知道它们存在"能帮你在读到怪代码时不当场懵住；更重要的是，想在类体内引用前面定义的名字，直接裸用即可（如 `X = SOMETHING + 1` 里引用类外的 `SOMETHING` 全局）。

·初学者误区：① 在方法里裸写类属性名——找不到；② 以为类体内引用"类自身的名字"（如 `class C: data = C.something`）可行——C 此时还未创建，会报 `NameError`；③ 以为 global 声明的效果会在类之间"传染"——每个类体、每个函数体的声明都是局部的。

29.17 属性名：对象命名空间 (Attribute Names: Object Namespaces)

英文原文

We've also seen that qualified attribute names refer to attributes of specific objects and obey the rules for

modules and classes. For class and instance objects, the reference rules are augmented to include the inheritance search procedure:-

- Assignment (`object.X = value`) — Creates or alters the attribute name `X` in the namespace of the object being qualified, and none other. Inheritance-tree climbing happens only on attribute reference, not on attribute assignment.
- Reference (`object.X`) — For class-based objects, searches for the attribute name `X` in `object`, then in all accessible classes above it, using the inheritance search procedure. For nonclass objects such as modules, fetches `X` from `object` directly.

As noted earlier, the preceding captures the normal case for typical code, but these attribute rules can vary in classes that utilize more advanced tools you'll meet later. For example, reference inheritance can be richer than implied here when metaclasses are deployed, and classes that leverage attribute management tools such as properties, descriptors, and `setattr` can intercept and route attribute assignments arbitrarily. In fact, some inheritance is run on assignment, too, to locate descriptors with a `set` method; such tools override the normal rules for both reference and assignment. We'll explore attribute management tools in depth in Chapter 38 and formalize inheritance and its use of descriptors in Chapter 40. For now, most readers should focus on the normal rules given here, which cover most Python application code you're likely to read, write, or run.

中文翻译

我们同样已经看到，限定的属性名引用的是特定对象的属性，并遵循模块和类的规则。对于类和实例对象，引用规则被增强，加入了继承查找过程：

- 赋值 (`object.X = value`) ——在被限定的 `object` 的命名空间中创建或改变属性名 `X`，仅此而已。继承树的攀爬只发生在属性引用时，而不发生在属性赋值时。
- 引用 (`object.X`) ——对基于类的对象，使用继承查找过程在 `object` 中、然后在它之上的所有可达类中搜索属性名 `X`。对非类对象（如模块），直接从 `object` 获取 `X`。

如前所述，以上描述捕获的是典型代码的常规情况，但在使用你以后会遇到的高级工具的类中，这些属性规则可能会变化。例如，部署元类时引用继承可以比这里暗示的更丰富；而使用属性（`properties`）、描述符（`descriptors`）和 `setattr` 等属性管理工具的类，可以随意拦截和路由属性赋值。事实上，一些继承在赋值时也会运行，目的是定位带 `set` 方法的描述符；这类工具覆盖了引用和赋值两方面的常规规则。我们将在第 38 章深入探讨属性管理工具，并在第 40 章正式化继承及其对描述符的使用。就目前而言，大多数读者应专注于这里给出的常规规则，它覆盖了你可能阅读、编写或运行的大部分 Python 应用代码。

深度理解

- 核心概念：限定名的两条铁律——赋值只看当前对象（写入 `object.dict`），引用才沿继承树向上。模块对象更简单：赋值/引用都只在一个字典里，无继承。
- 底层实现：CPython 里 `object.X = v` 最终调用 `PyObject_SetAttr`，默认路径就是 `type(ob)->tpsetattr`（对普通

类对象就是"写实例字典")；object.X 走 tpgetattro → 实例字典 → 类型字典 (MRO)。描述符 (含 property) 能插足是因为它们在查找过程中有优先权 (数据描述符优先于实例字典)。

·为什么这样设计：如果赋值也爬树，就没办法"在实例上遮蔽类属性"了——遮蔽 (shadowing) 是 OOP 动态语言的基本能力。固定"写只写眼前"，规则简单且功能完备。

·实际问题：调试"为什么改了没生效"时，先问一句：我是通过赋值还是引用操作的？引用会爬树、赋值不会——80% 的诡异 bug 都源于这两者之一用错了对象。

·初学者误区：① 认为 `self.x = 1` 会把 `x` 放进"类里"——不，放进了实例字典；② 认为模块也有继承——没有，`module.attr = v` 永远是本模块字典；③ 被 `property/descriptor` 拦路的"意外"——这些是后话，但现在记住"有例外"这个提醒，将来遇到不按规则出牌的行为就不会惊掉下巴。

29.18 命名空间的"禅意"：赋值给名字分类 (The "Zen" of Namespaces: Assignments Classify Names)

英文原文

With distinct search procedures for qualified and unqualified names, and multiple lookup layers for both, it

can sometimes be difficult to tell where a name will wind up going. In Python, the place where you assign a name is crucial—it fully determines the scope or object in which a name will reside. The file in Example 29-2, `manynames.py`, illustrates how this principle translates to code and summarizes the namespace ideas we have seen throughout this book (sans obscure special-case scopes like comprehensions):

中文翻译

由于限定名和非限定名有着不同的查找过程，两者又都有多个查找层次，有时很难说清一个名字最终会落在哪里。在 Python 中，你赋值名字的位置至关重要——它完全决定了名字将栖身于哪个作用域或对象。示例 29-2 中的文件 `manynames.py` 说明了这个原则如何转化为代码，并总结了我们在本书中见过的命名空间思想（不包括推导式之类的晦涩特例作用域）：

代码分析

Example 29-2. `manynames.py` (前半部分)

```
X = 11                                # X manynames.X

def f():
    print(X)                          # LEGB X

def g():
    X = 22                            # X X
    print(X)

class C:
```

```

X = 33                # C.X self.m() self.X
def m(self):
    X = 44            # X
    self.X = 55       # self.X X

```

解释器如何处理这段代码：

1. 五个 X，五个世界：X = 11 写入模块字典；f 里的 print(X) 编译为 LOADGLOBAL（模块级）；g 里的 X = 22 编译为 LOADFAST/STOREFAST（函数局部）；C 体内的 X = 33 落入类字典；m 里的 X = 44 是方法局部（与 self.X 毫无关系）。

2. 编译器如何区分：编译器逐代码对象地分析"这个作用域里有没有对 X 的赋值"——有，X 就是局部的（LOADFAST）；没有，就当作全局或闭包。名字的归属在编译期就被钉死，运行时只是执行既定指令。

3. self.X = 55 的特别之处：self 是参数（局部），X 是属性名——这条语句写入的是"实例对象的字典"，与作用域机制完全无关。这就是"一个名字五种命运"的完整图解。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

This file assigns the same name, `X`, five times—illustrative, though not exactly best practice! Because this name is assigned in five different locations, though, all five `X`s in this program are completely different variables. From top to bottom, the assignments to `X` here generate a module attribute (11), a local variable in a function (22), a class attribute (33), a local variable in a method (44), and an instance attribute (55). Although all five are named `X`, the fact that they are all assigned at different places in the source code or to different objects makes all of these unique variables. You should study this example carefully because it collects ideas we've been exploring throughout the last few parts of this book. When it makes sense to you, you will have achieved Python namespace enlightenment. Or you can run the code and see what happens—Example 29-3 lists the remainder of the source file in Example 29-2, with self-test code that makes an instance and prints all the `X`s that it can fetch.

中文翻译

这个文件把同一个名字 `X` 赋值了五次——有教学意义，但绝不是最佳实践！不过，正因为这个名字被赋在五个不同的位置，程序中这五个 `X` 是完全不同的变量。从上到下，这里的 `X` 赋值依次生成了：一个模块属性（11）、一个函数中的局部变量（22）、一个类属性（33）、一个方法中的局部变量（44）和一个实例属性（55）。虽然五个都叫 `X`，但它们在源码中的赋值位置不同、或赋给了不同的对象，这一事实使它们全都是独一无二的变量。你应该仔细研究这个例子，因为它汇集了我们在这本书最近几个部分一直在探索的思想。当你觉得它说得通了，你就达成了 Python 命名空间的开悟（namespace enlightenment）。或者你也可以运行代码看看会发生什么——示例 29-3 列出了示例 29-2 中源文件的剩余部分，其中的自测代码创建一个实例并打印它能获取的所有 `X`。

代码分析

Example 29-3. `manynames.py`（后半部分）

```
if __name__ == '__main__':
    print(X)           # 11 manynames.X
    f()                # 11
    g()                # 22
    print(X)           # 11
    I = C()            #
    print(I.X)          # 33
    I.m()              # X
    print(I.X)          # 55
    print(C.X)          # 33 I X I.X
    #print(C.m.X)       #
    #print(g.X)         #
```

解释器如何处理这段代码：

1. 继承查找的两次结果不同：print(l.X) 在调用 l.m() 之前输出 33（实例字典为空 → 两步协议走到类字典命中 C.X）；l.m() 执行后 self.X = 55 写进实例字典，再 print(l.X) 第一步就命中 55。同一个表达式，不同时刻，不同结果——属性查找是运行时行为，这就是动态性的活教材。
2. 两个被注释的"失败"：C.m.X——m 里的 X 是方法局部变量，类字典里根本没有；g.X——函数局部变量在函数栈帧里，对象化访问无从谈起。局部变量只在调用期间存在于栈帧（frame.flocals）中，函数返回即销毁。
3. name == 'main' 门卫：被导入时此段不执行（模块名是 manynames），作为脚本运行时才执行——这是自测代码的标准挂载点。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

The outputs that are printed when the file is run are noted in the comments in the code; trace through them to see which variable named X is being accessed each time. Notice in particular that we can go through the class to fetch its attribute (C.X), but we can never fetch local variables in functions or methods from outside their def statements. Locals are visible only to other code within the def, and, in fact, only live in memory while a call to the function or method is executing. Some of the names defined by this file are visible outside the file to other modules, too, but recall that we must always import before we can access names in another file—name segregation is the main point of modules, after all. Example 29-4 shows how names appear outside the module in Example 29-3, again with expected outputs in comments:

中文翻译

文件运行时的打印输出已标注在代码的注释中；请逐一跟踪，看每一次访问的是哪个名叫 X 的变量。特别注意：我们可以通过类来获取它的属性 (C.X)，但永远无法从 def 语句之外获取函数或方法中的局部变量。局部变量只对 def 内部的其他代码可见，而且事实上，只有在调用函数或方法期间，它们才存在于内存中。这个文件定义的一些名字在文件外对其他模块也是可见的，但请记住：我们必须先导入才能访问另一个文件中的名字——毕竟，名字隔离正是模块

的主要意义。示例 29-4 展示了示例 29-3 中的名字在模块外部如何呈现，同样带注释的期望输出：

代码分析

Example 29-4. manynames-client.py

```
import manynames
X = 66
print(X)                # 66
print(manynames.X)      # 11
manyname.f()            # 11manyname X
manyname.g()            # 22
print(manyname.C.X)     # 33
I = manyname.C()
print(I.X)              # 33
I.m()
print(I.X)              # 55
```

解释器如何处理这段代码：

1. 导入即命名空间切换：import manynames 创建模块对象；manyname.X 是"限定名"，查的是模块字典——与客户端自己的 X = 66 井水不犯河水。这就是"作用域由源码位置决定，与导入关系无关"（lexical 原则）的证明。
2. 跨文件函数调用：manyname.f() 执行的是 manyname 模块里的函数体——它的 print(X) 按它自己的作用域链解析（manyname 模块的全局），输出 11，而不是调用方的 66。函数记住的是定义它的环境，不是调用它的环境（闭包精神的极简版）。
3. 类/实例访问跨模块照常：manyname.C.X、I.X 的查

找协议与同文件内完全一致——继承查找不看模块边界，只看对象链接。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Notice here how `manynames.f()` prints the `X` in many names, not the `X` assigned in this file—scopes are always determined by the position of assignments in your source code (i.e., lexically) and are never influenced by what imports what or who imports whom. Also, notice that the instance's own `X` is not created until we call `l.m()`—attributes, like all variables, spring into existence when assigned, and not before. Normally, we create instance attributes by assigning them in class init constructor methods, but this isn't the only option. Finally, as covered in Chapter 17, it's also possible

for a function to change names outside itself with `global` and `nonlocal` statements—these statements provide write access, but also modify assignment's namespace binding rules. Example 29-5 provides a refresher on these points.

中文翻译

注意这里 `manynames.f()` 打印的是 `manynames` 里的 `X`，而不是本文件中赋值的 `X`——作用域总是由源码中赋值的位置决定（即词法上决定），从不被“谁导入什么、谁被谁导入”所影响。另外注意，实例自己的 `X` 直到我们调用 `l.m()` 才被创建——属性像所有变量一样，在赋值时才凭空出现，不会更早。通常，我们在类的 `__init__` 构造方法中通过赋值创建实例属性，但这不是唯一选项。最后，正如第 17 章所讲的，函数还可以用 `global` 和 `nonlocal` 语句改变自身之外的名字——这些语句提供写访问权限，同时也修改了赋值的命名空间绑定规则。示例 29-5 提供了这些要点的温习。

代码分析

Example 29-5. `funcscope.py`

```
X = 11                                #
def g1():
    print(X)                          # 11
def g2():
    global X                          #
    X = 22                            #
def h1():
    X = 33                            #
    def nested():
```

```

        print(X)           # 33
def h2():
    X = 33                 #
    def nested():
        nonlocal X        #
        X = 44            #

```

解释器如何处理这段代码：

1. global 改变赋值去向：g2 里若没有 global X, X = 2 会创建一个函数局部；有了声明，STOREGLOBAL 指令把 22 写进模块字典——即"引用（读）是 LEGB，写权限（写）是声明授予的"。
2. nonlocal 打通闭包写路径：h2.nested 里若没有 nonlocal X, X = 44 是 nested 自己的局部；有了声明，STOREDEREF 指令写入 h2 栈帧里的闭包变量（closure 指向的 cell）——注意这里更新的是 h2 的 X = 33，模块级的 X = 11 毫发无损。
3. 闭包读路径：h1.nested 的 print(X) 编译为 LOADDEREF，从闭包 cell 读取 33——nested 之所以记得 X，是因为它引用了外层函数局部（cell 化机制），而非"复制"了值。

深度理解

·核心概念：一句话禅意——"名字的命运由赋值位置决定"（Assignments classify names）。同一个 X 赋在模块顶层、函数内、类体内、方法内、self. 上，就是五个完全不同的变量。加上"读按 LEGB/继承树找、写永远写当前作用域/对象"两条规则，整个名字系统就齐了。

·底层实现：编译器在编译每个代码对象时先做"符号表分析" (symbol table) , 找出作用域内被赋值的名字 → 标记为局部 (FAST) ; 被 global/nonlocal 声明的 → 标记为模块 (GLOBAL) 或闭包 (DEREF) 。一切在编译期定型, 运行时只是机械执行。

·为什么这样设计：静态决定名字归属, 让"哪个名字是哪个变量"可以靠读代码确定 (哪怕跨文件) , 程序可读性/可调试性大增; 这是 Python 刻意区别于 Perl/JavaScript 等"运行时才见分晓"动态语言的地方。

·实际问题：代码评审时看到同名变量满天飞——用这条原则快速圈定每个名字的"户籍"; 调试时看到"奇怪的值", 先问：这个 X 在哪个作用域有赋值？

·初学者误区：① 以为类体赋值和方法内 self.X 是同一个东西 (不是) ; ② 以为 import 会"共享"全局变量 (不会, 每个模块有自己的字典, 改 manynames.X 要显式通过 manynames. 前缀) ; ③ 以为不加 global 也能"顺手修改全局变量"——读可以, 写必须先声明, 否则静默创建一个同名局部, 这是最隐蔽的 bug 源之一。

29.19 嵌套类：重温 LEGB 作用域规则 (Nested Classes: The LEGB Scopes Rule Revisited)

英文原文

The preceding example summarized the effect of nested functions on scopes, which we studied in Chapter 17. As we saw briefly near the start of this section, classes can be nested, too—a useful coding pattern in some types of programs. This has scope implications that follow naturally from what you already know, but that may not be obvious on first encounter. This section illustrates the concept by example. Though they are normally coded at the top level of a module, classes also appear nested in functions that generate them—a variation on the "factory function" (a.k.a. closure) theme in Chapter 17, with similar state retention roles. There, we noted that class statements introduce new local scopes, much like function `def` statements, which follow the same LEGB scope lookup rule as function definitions. This rule applies both to the top level of the class itself as well as to the top level of method functions nested within it. Both form the L layer in this rule—they are local scopes with access to their names, names in any enclosing functions, globals in the enclosing module, and built-ins. Like modules, the class's local scope morphs into an attribute namespace after the class statement is run, but its top-level code is a local scope while the class runs. Importantly, though, although classes have access to enclosing functions' scopes, they do not themselves act as enclos

ing scopes to code nested within the class—Python searches enclosing functions for referenced names but never any enclosing classes. That is, a class is a local scope and has access to enclosing local scopes, but it does not serve as an enclosing local scope to further nested code. Because the search for names used in method functions skips the enclosing class, class attributes must be fetched as object attributes using inheritance. For example, in the nester function of Example 29-6, all references to X are routed to the global scope except the last, which picks up a local-scope redefinition in method2 (the output of each example in this section is described in its last two comments).

中文翻译

前面的例子总结了嵌套函数对作用域的影响，这是我们在第 17 章研究过的。正如本节开头我们简要看到的那样，类也可以被嵌套——在某些类型的程序中这是一个有用的编码模式。这会产生一些你基于已有知识能自然推导出、但初次遇到时可能不明显的作用域影响。本节通过例子来说明这个概念。虽然类通常编码在模块的顶层，但它们也会出现在生成它们的函数中——这是第 17 章“工厂函数” (factory function, 又名闭包 closure) 主题的变体，具有类似的状态保留 (state retention) 角色。在那里我们指出，class 语句会引入新的局部作用域，很像函数的 def 语句，并遵循与函数定义相同的 LEGB 作用域查找规则。这条规则既适用于类本身的顶层，也适用于嵌套在其中的方法函数的顶层。两者都构成该规则中的 L 层——它们都是局部作用域，可

以访问自己的名字、任何包围函数中的名字、包围模块中的全局量以及内建量。像模块一样，类的局部作用域在 `class` 语句运行之后就变形为属性命名空间，但在 `class` 运行期间，它的顶层代码是一个局部作用域。不过，重要的是：虽然类可以访问包围函数的作用域，但类本身不会对类内嵌套的代码充当包围作用域——Python 会搜索包围函数来查找引用的名字，但从不搜索任何包围的类。也就是说，类是局部作用域、能访问外层局部作用域，但它不充当进一步嵌套代码的包围局部作用域。因为方法函数中名字的查找会跳过包围它的类，所以类属性必须通过继承——以对象属性的方式——来获取。例如，在示例 29-6 的 `nester` 函数中，对 `X` 的所有引用都被路由到全局作用域，只有最后一个除外，它在 `method2` 中捡到了一个局部作用域的重定义（本节每个例子的输出都描述在其最后两条注释中）。

代码分析

Example 29-6. `classscope1.py`

```
X = 1
def nester():
    print(X)                # 1
    class C:
        print(X)            # 1
        def method1(self):
            print(X)         # 1
        def method2(self):
            X = 3             #
            print(X)         # 3
    I = C()
    I.method1()
    I.method2()
print(X)                    # 1
```

解释器如何处理这段代码：

1. 类体是一个 L 层：class C: 的顶层 print(X) 编译为 LOADGLOBAL——因为类体里没有 X 的赋值，X 一路顺着 L（类体）→ E（无包围函数到外层？不对——nester 是包围函数，但顶层直接 print）→ G（模块）命中 X = 1。类体本身是局部作用域，但对没有在类体赋值的名字，它和任何局部作用域一样向上找。
2. 方法跳过类体直接看外层：method1 里 print(X) → LOADGLOBAL，输出 1。注意它没有先看 class C 体的作用域——类被排除在 E 之外。若有类属性 C.Y，方法里要写 self.Y 或 C.Y。这里 method2 定义了自己的 X = 3，所以 method2 里的 print(X) 是本方法局部（3）。
3. 输出序列：nester() 运行：顶层 print → 1；创建 C 时类体 print → 1；method1 → 1；method2 → 3。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Watch what happens, though, when we reassign the same name in nested function layers in Example 29-7 : the redefinitions of X create locals that hide those in enclosing scopes, just as for simple nested functions; the enclosing class layer does not change this rule, and in fact is irrelevant to it.

中文翻译

不过，看看在示例 29-7 中，当我们在嵌套函数层中重新赋值同一个名字时会发生什么：X 的重定义创建了局部，隐藏了包围作用域中的那些——就像简单的嵌套函数一样；外围的类层不改变这条规则，事实上与它无关。

代码分析

Example 29-7. classscope2.py

```
X = 1
def nester():
    X = 2                #
    print(X)             # 2
    class C:
        print(X)         # defnester2
        def method1(self):
            print(X)      # defnester2
        def method2(self):
            X = 3         # nester
            print(X)      # 3
```

```
I = C()
I.method1()
I.method2()

print(X)           # 1
nester()           # 2, 2, 2, 3
```

解释器如何处理这段代码：

1. 类体看到了包围函数：nester 里有 $X = 2$ ，所以 nester 的 X 是它的局部。类体 class C: 的顶层 print(X) 由于类体未赋值 X ，进入 LEGB 的 E 层——找到 nester 的局部 2，输出 2。类体会被包进 enclosing function 的闭包作用域。
2. 方法同样继承这个"视线"：method1 的 print(X) $\rightarrow$ E 层命中 nester 的 2。method2 自己 $X = 3$ ，所以它是 3。
3. 关键区别：类体和方法能看到的是外层函数的局部，而看不到外层类的局部——nester 是函数（在 E 链上），class C 不是（不在 E 链上）。这正是"LEGB 不存在，只有 LEGB"（作者原话: it's the LEGB rule, not LCEGB!）的实证。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

Finally, Example 29-8 shows what happens when we reassign the same name at multiple stops along the way: assignments in the local scopes of both functions and classes hide globals or enclosing function locals of the same name, regardless of the nesting involved.

中文翻译

最后，示例 29-8 展示了当我们在沿途多个位置重新赋值同一个名字时会发生什么：函数和类的局部作用域中的赋值都会隐藏同名的全局变量或包围函数的局部变量，无论嵌套如何。

代码分析

Example 29-8. classscope3.py

```
X = 1
def nester():
    X = 2                #
    print(X)             # 2
    class C:
        X = 3            # nester C.X I.X
        print(X)         # 3
        def method1(self):
            print(X)      # def 32
```

```

        print(self.X) # 3
    def method2(self):
        X = 4          # nester
        print(X)       # 4
        self.X = 5     #
        print(self.X) # 5

I = C()
I.method1()
I.method2()

print(X)              # 1
nester()              # 2, 3, 2, 3, 4, 5

```

解释器如何处理这段代码：

1. 类体重定义只影响"属性"不影响"作用域"：class C 顶层 `X = 3` 把 3 存入类字典，所以类体的 `print(X)`（裸名，未赋值？不——类体有 `X=3` 赋值，所以类体里 `X` 是类体局部）输出 3；`method1` 里的裸 `X` 却在 E 层命中 `nester` 的 2——类体的 `X=3` 对方法的作用域链毫无影响。
2. 两条路取类/实例数据：要让方法"看到"类的 3 或实例的 5，唯一的通道是 `self.X`（实例属性）或 `C.X`（类属性）这种限定访问——它们在类字典/实例字典中查找，不走作用域链。`self.X` 在 `method1` 中命中继承来的类属性 3；`method2` 中 `self.X = 5` 写入实例字典后 `print(self.X)` 第一步命中 5。
3. 输出序列：`nester` 顶层 2 → 类体 3 → `method1` 裸 `X=2`、`self.X=3` → `method2` 裸 `X=4`、`self.X=5`。

深度理解

·核心概念：LEGB 里没有 C——类既不是方法名字的包围作用域（E），也不会成为方法名字的归宿。一图流记忆

：method1 里的裸 X 永远跳过 class C 体内的 X；要拿类/实例数据，必须走 self.X 或 C.X。

·底层实现：方法函数是独立代码对象，它的自由变量 (free variable, 裸名 X) 来自"定义它的外层函数作用域" (编译器的 scoping 分析确定 nested closure cells)；类对象字典不在这个链上。self.X 走属性查找，X 走栈与 cell 链——两条完全不同的数据通路。

·为什么这样设计："类不参与 LEGB"避免了最混乱的歧义——如果方法里的裸 X 会在类体里找，那么同名的类属性、全局变量、方法局部将形成三向纠缠，程序员无法静态判断结果。一刀切掉 C，规则回归单纯。

·实际问题：这就是为什么方法里的实例数据必须 self.xx——不是惯例，是语法使然；也是为什么"类体顶层可以 print(classvar) 直接裸用类变量、而方法里不行"——前者类体是作用域，后者方法的作用域链里没有类。

·初学者误区：① 在方法里裸用类变量名报 NameError 时百思不解；② 以为"类体里定义的变量在方法里自动可见"——错误；③ 把"嵌套类"误认为与 Java 内部类的闭包行为相同——Python 里类体会"向外看"到外层函数，但方法不会"向中看"到类体。

29.20 命名空间字典：回顾 (Namespace Dictionaries: Review)

英文原文

In Chapter 23, we saw that module namespaces have a concrete implementation as dictionaries, exposed with the built-in `dict` attribute. In Chapters 27 and 28, we saw that the same holds true for class and instance objects—attribute qualification is mostly a dictionary indexing operation internally, and attribute inheritance is largely a matter of searching linked dictionaries.

In fact, within Python, instance and class objects are mostly just dictionaries with links between them. Python exposes these dictionaries, as well as their links, for use in advanced roles.

We put some of these tools to work in the prior chapter, but to summarize and help you better understand how attributes work internally, let's work through an interactive session that traces the way namespace dictionaries grow when classes are involved. Now that we know more about methods and superclasses, we can also embellish the coverage here for a better look. First, let's define a superclass and a subclass with methods that will store data in their instances:

```
>> class Super:
def hello(self): self.data1 = 'hack'>> >> class Sub(Super):
```

`def hola(self): self.data2 = 'code'` When we make an instance of the subclass, the instance starts out with an empty namespace dictionary, but it has links back to the class for the inheritance search to follow. In fact, the inheritance tree is explicitly available in special attributes, which you can inspect. Instances have a `class` attribute that links to their class, and classes have a `bases` attribute that is a tuple containing links to higher superclasses:

```
>> X = Sub() >> X.dict # Instance namespace dict {}
>> X.class # Class of instance

<class'main.Sub'> >> Sub.bases # Superclasses of class
(<class'main.Super'>,) >> Super.bases # Implied
above top-levels (<class'object'>,)

```

As classes assign to `self` attributes, they populate the instance objects—that is, attributes wind up in the instances' attribute namespace dictionaries, not in the classes'. An instance object's namespace records data that can vary from instance to instance, and `self` is a hook into that namespace:

```
>> Y = Sub() >> X.hello() >> X.dict {'data1':'hack'} >
> X.hola() >> X.dict {'data1':'hack','data2':'code'} >> list(Sub.dict.keys())
['module','hola','doc'] >> list(Super.dict.keys())
['module','hello','dict','weakref','doc'] >> Y.dict {}

```

Notice the extra underscore names in the class dictionaries; Python sets these automatically, and we can fi

Iter them out with the generator expressions we coded in Chapters 27 and 28 omitted here for space. Most are not used in typical programs but may be used by tools (e.g., doc holds the docstrings discussed in Chapter 15).

Also, observe that Y, a second instance made at the start of this series, still has an empty namespace dictionary at the end, even though X's dictionary has been populated by assignments in methods. Again, each instance has an independent namespace dictionary, which starts out empty and can record completely different attributes than those recorded by the namespace dictionaries of other instances of the same class.

Because instance attributes are actually dictionary keys inside Python, there are really two ways to fetch and assign their values—by qualification or by key indexing:

```
>> X.data1, X.dict['data1'] ('hack','hack') >> X.data3  
='docs'>> X.dict {'data1':'hack','data2':'code','data3':  
docs'} >> X.dict['data3'] ='apps'>> X.data3'apps'
```

This equivalence applies only to attributes actually attached to the instance, though. Because attribute fetch qualification also performs an inheritance search, it can access inherited attributes that namespace dictionary indexing cannot. The inherited attribute `X.hello`, for instance, cannot be accessed by `X.dict['hello']`.

Experiment with these special attributes on your own to get a better feel for how namespaces actually do their attribute business. Also, try running these objects through the `dir` function we met in the prior two chapters—`dir(X)` is similar to `X.dict.keys()`, but `dir` sorts its list and includes inherited attributes. Even if you will never use these in the kinds of programs you write, seeing how attributes are stored can help solidify namespaces in general.

中文翻译

在第 23 章，我们看到模块命名空间有作为字典的具体实现，通过内建的 `dict` 属性暴露出来。在第 27、28 章，我们看到类和实例对象也是如此——属性限定在内部大部分是一个字典索引操作，属性继承在很大程度上是搜索链接字典的问题。事实上，在 Python 内部，实例对象和类对象大多只是带链接的字典。Python 把这些字典以及它们的链接都暴露出来，供高级用途使用。我们在前一章已经把这些工具用了一部分，但为了总结并帮助你更好地理解属性在内部如何工作，让我们完整走一遍交互式会话，跟踪涉及类时命名空间字典是如何增长的。现在我们对方法和超类知道得更多了，也可以在这里补充更深入的讲解。首先，让我们定义一个超类和一个子类，它们的方法会把数据存进各自的实例：

```
>>> class Super: def hello(self): self.data1 = 'hack'>>>
>>> class Sub(Super): def hola(self): self.data2 = 'code'
```

当我们创建子类的实例时，实例一开始带有一个空的命名空间字典，但它有指向类的链接，供继承查找使用。事实上，继承树就显式地存在于特殊属性中，你可以检查它们。

实例有 class 属性链接到它的类，类有 bases 属性——一个包含指向更高层超类链接的元组：>>> X = Sub() >>> X.dict # 实例命名空间字典{} >>> X.class # 实例的类<class'main.Sub'> >>> Sub.bases # 类的超类(<class'main.Super'>,) >>> Super.bases # 顶层之上隐式的(<class'object'>,) 当类给 self 属性赋值时，它们是在填充实例对象——也就是说，属性会落进实例的属性命名空间字典，而不是类的。实例对象的命名空间记录的是各实例之间可能不同的数据，而 self 就是通向那个命名空间的钩子：>>> Y = Sub() >>> X.hello() >>> X.dict {'data1':'hack'} >>> X.hola() >>> X.dict {'data1':'hack','data2':'code'} >>> list(Sub.dict.keys()) ['module','hola','doc'] >>> list(Super.dict.keys()) ['module','hello','dict','weakref','doc'] >>> Y.dict {} 注意类字典中那些额外的下划线名字；Python 自动设置它们，我们可以用第 27、28 章编写的生成器表达式把它们过滤掉（此处为节省篇幅省略）。它们大多在典型程序里用不到，但可被工具使用（例如 doc 保存的是第 15 章讨论过的文档字符串）。另外注意，Y——这一系列操作开始时创建的第二实例——到最后仍然有一个空的命名空间字典，尽管 X 的字典已被方法中的赋值填满。再次强调：每个实例都有独立的命名空间字典，它从空开始，可以记录与同类其他实例完全不同的属性。因为实例属性在 Python 内部实际上就是字典键，所以获取和赋值它们有两种方式——通过限定或通过键索引：>>> X.data1, X.dict['data1'] ('hack','hack') >>> X.data3 = 'docs' >>> X.dict {'data1':'hack','data2':'code','data3':'docs'} >>> X.dict['data3'] = 'apps' >>> X.data3 'apps' 不过，这种等价关系只适用于实际挂靠（attach）在实例上的属性。因为属性

获取的限定操作还会执行继承查找，它能访问命名空间字典索引无法访问的继承属性。例如，继承来的属性 `X.hello` 就不能通过 `X.dict['hello']` 访问。请自己动手实验这些特殊属性，更好地感受命名空间实际上是如何处理属性事务的。另外，试着用我们前两章见过的 `dir` 函数处理这些对象——`dir(X)` 类似于 `X.dict.keys()`，但 `dir` 会对列表排序并包含继承属性。即使你永远不会在自己写的这类程序中使用它们，看到属性是如何存储的也有助于巩固对命名空间的整体理解。

深度理解

·核心概念：“类与实例 = 带链接的字典”。`dict`、`class`、`bases` 是解释器内部结构的“窗户”——继承树、命名空间字典都看得见摸得着。`dir()` 则是“带继承的排序版 `dict`”。

·底层实现：CPython 中实例的类型 `PyObject + dict` (`PyObjectGenericGetAttr` 先查实例字典再查类型字典)；类对象的 `dict` 是映射代理 (`mappingproxy`，只读视图)。slots (第 32 章) 会移除实例字典，这正是“属性可能不在字典里”的例外来源。

·为什么这样设计：把属性机制完全建立在 Python 自己的字典之上（而不是另造一套内部结构），带来两大好处：透明可检查（调试神器）、可用普通代码生成/遍历（工具编写友好）。

·实际问题：三件套用途——遍历对象属性、序列化（把 `instance.dict` 存成 JSON）、通用调试工具（打印对象的所有字段）；类树可视化就是 29.21 节(`classtree`)的玩法。

·初学者误区：① 用 `X.dict['hello']` 想拿继承来的方法——报 `KeyError`，因为方法根本不在实例字典里，只能在类字典里找；② 以为 `dir()` 就是“实例专属属性列表”——它包含类/超类的一切（排序后）；③ 忘记 `class.dict` 是只读映射代理，想 `Class.dict['x'] = 1` 会失败。

NOTE (slots 例外)：在第 32 章你将学到 slots——一个把属性存在实例中但不存在其命名空间字典里的高级类工具。把它们当作类属性很有诱惑力，事实上它们确实出现在类命名空间中并管理每实例的值。但你会发现，slots 可能阻止实例创建 dict——这是通用工具有时必须考虑到的隐患，办法是用存储无关 (storage-neutral) 的内建函数，比如用 `dir` 列举、用 `getattr` 获取。好消息是 slots 很少被使用——理应如此！

29.21 命名空间链接：爬树工具 (Namespace Links: A Tree Climber)

英文原文

The prior section demonstrated the special class and bases instance and class attributes without really explaining why you might care about them. In short, these attributes allow you to inspect inheritance hierarchies within your own code. For example, they can be used to display a class tree, as coded in the module of Example 29-9.

中文翻译

前一节演示了特殊的实例和类属性 `class` 和 `bases`，但没有真正解释你为什么会在意它们。简而言之，这些属性允许你在自己的代码中检查继承层级。例如，它们可以用来显示一棵类树（class tree），如同示例 29-9 的模块中所编码的那样。

代码分析

Example 29-9. classtree.py

```
"""
classtree.py:

"""

def classtree(cls, indent):
    print('.' * indent + cls.__name__) #
    for supercls in cls.__bases__:    #
        classtree(supercls, indent+3) #

def instancetree(inst):
    print('Tree of', inst)            #
    classtree(inst.__class__, 3)      #

def selftest():
    class A: pass
    class B(A): pass
    class C(A): pass
    class D(B,C): pass
    class E: pass
    class F(D,E): pass
    instancetree(B())
    instancetree(F())
```

```
if __name__ == '__main__': selftest()
```

解释器如何处理这段代码：

1. 递归遍历：classtree(cls, indent) 打印当前类名（缩进为 indent 个点），然后对 cls.bases 里的每个超类调用自己，缩进加深 3。像第 19 章的递归一样，每一层调用都有自己的 cls 和 indent 副本（调用栈上的独立帧）。当 bases 为空（如 object）或为空元组时，for 循环结束，递归自然到底。
2. 入口函数：instancetree(inst) 从实例出发：打印实例，再 classtree(inst.class, 3)——利用 class 指针把"实例世界"接到"类世界"。
3. 菱形会重复访问：class D(B, C)、B/C 都继承 A 时，A 会被打印两次（分别经 B 和 C 两条路径）——注释"may visit super > once"指的就是这个，它如实呈现了继承树的结构而非 MRO 线性序列。
4. object 现身：顶层类（如 A、E）的 bases 是 (<class'object'>,)，所以每个根节点之下都会自动带出 object——这是 Python 3 中"一切类默认继承 object"的直观证明。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

The classtree function in this script is recursive—it prints a class's name using name, then climbs up to the superclasses by calling itself. This allows the function to traverse arbitrarily shaped class trees; the recursion climbs to the top and stops at root superclasses that have empty bases attributes. As explored in Chapter 19, when using recursion, each active level of a function gets its own copy of the local scope. Here, this means that cls and indent are different at each classtree level. Most of this file is self-test code. When run standalone, it builds an empty class tree, makes two instances from it, and prints their class tree structures. The trees include the implied object superclass that is automatically added above standalone root (i.e., to pmost) classes; there's more on object in Chapter 32:

```
$ python3 classtree.py Tree of <main.selftest.<locals>
>.B object at 0x10733a000> ...B .....A .....object Tree
of <main.selftest.<locals> .F object at 0x10733a000> .
..F .....D .....B .....A .....object .....C .....A .
.....object .....E .....object Here, indentation mar
```

ked by periods is used to denote class tree height. Of course, we could improve on this output format and perhaps even sketch it in a GUI display. Even as is, though, we can import these functions anywhere we want a quick display of a physical class tree: `$ python3`

```
>>> class Employee: pass
>>> class Person(Employee): pass
>>> pat = Person()
>>> import classtree
>>> classtree.instancetree(pat)
Tree of <main.Person object at 0x1072a1b80>
...Person .....Employee .....object
```

Regardless of whether you will ever code or use such tools, this example demonstrates one of the many ways that you can make use of special attributes that expose interpreter internals. You'll see others when we code general-purpose class display tools in "Multiple Inheritance and the MRO"—there, we will extend this technique to also display attributes in each object in a class tree and function as a reusable superclass. In the last part of this book, we'll revisit such tools in the context of Python tool building at large, to code tools that implement attribute privacy, argument validation, and more. While not in every Python programmer's job description, access to internals enables powerful development tools.

中文翻译

这个脚本中的 `classtree` 函数是递归的——它用 `name` 打印类的名字，然后通过调用自身向上爬向超类。这让函数能够遍历任意形状的类树；递归一直爬到顶端，在拥有空 `bas`

es 属性的根超类处停止。正如第 19 章探讨过的，使用递归时，函数每活跃一层都有自己的一份局部作用域副本。这里意味着，每一层 classtree 的 cls 和 indent 都是不同的。这个文件大部分是自测代码。独立运行时，它构建一棵空类树，从中创建两个实例，并打印它们的类树结构。这些树包含那个隐式的 object 超类——它被自动添加到独立的根（即最顶层）类之上；关于 object 的内容在第 32 章更多：

```
$ python3 classtree.py Tree of <main.selftest.<locals>.B object at 0x10733a000> ...B .....A .....object
Tree of <main.selftest.<locals>.F object at 0x10733a000> ...F .....D .....B .....A .....object .....C .....
...A .....object .....E .....object
```

在这里，用句点标记的缩进用来表示类树的高度。当然，我们可以改进这种输出格式，甚至可以在 GUI 显示中把它勾勒出来。不过，即使保持原样，我们也可以在任何想要快速显示一棵物理类树的地方导入这些函数：

```
$ python3 >>> class Employee: pass
>>> class Person(Employee): pass
>>> pat = Person()
>>> import classtree
>>> classtree.instancetre
e(pat) Tree of <main.Person object at 0x1072a1b80>
...Person .....Employee .....object
```

无论你是否会编写或使用这样的工具，这个例子都演示了利用暴露解释器内部机制（interpreter internals）的特殊属性多种方式之一。当我们编写通用类显示工具时（在“Multiple Inheritance and the MRO”一节）你还会看到其他用法——在那里，我们将扩展此技术，在类树的每个对象中显示属性，并让它作为可复用的超类发挥作用。在本书最后一部分，我们将在更广阔的 Python 工具构建背景下重新审视这类工具，用它编写实现属性私有性、参数验证等等的工具。虽然这不属于每位

Python 程序员的职责描述，但对内部的访问（access to internals）让强大的开发工具成为可能。

深度理解

- 核心概念：让代码"看见自己"——通过 class、bases 在运行时重建并遍历继承树。这是"元编程"（metaprogramming）的敲门砖：程序可以检查、生成、修改自身结构。
- 底层实现：bases 是只读元组（保护类图在运行时不轻易被篡改），class 是每实例一个的指针；递归遍历时用"点 × 深度"表达树高，是控制台输出的经典技巧。object 出现在树底是因为每个默认类定义都是 class X(object) 的糖。
- 为什么这样设计：把"类图"在运行时暴露出来，而非锁死在编译期，使得反射（reflection）、调试器、可视化工具、ORM（对象关系映射）都能写出与具体类层级无关的通用逻辑。
- 实际问题：调试器打印"对象是谁的子类"、ORM 根据 class 判断表名、序列化库遍历属性.....这类"通用工具"全部依赖对 class/bases/dict 的读取。
- 初学者误区：① 以为继承树是不可见的抽象——它其实可遍历可打印；② 以为 bases 是"所有祖先列表"——它只是直接父类元组，要祖先得递归或看 mro；③ 以为这类工具与日常编程无关——仅调试就够你受用无穷（pdb/dir/格式化打印全靠这些内部结构）。

29.22 重温文档字符串 (Documentation Strings Revisited)

英文原文

The last section's example includes a docstring for its module, but remember that docstrings can be used for class components as well. Docstrings, which we covered in detail in Chapter 15, are string literals that show up at the top of various structures and are automatically saved by Python in the corresponding objects' doc attributes. This works for module files, function defs, and classes and methods. Now that we've seen more about classes and methods, Example 29-10, a.k.a. docstr.py, provides a quick but comprehensive example that summarizes the places where docstrings can show up in your code. All of these can be triple-quoted blocks or simpler one-liner literals like those here.

中文翻译

上一节的示例为其模块包含了一个文档字符串 (docstring)，但请记住，文档字符串也可以用于类组件。文档字符串（第 15 章详细讨论过）是出现在各种结构顶部的字符串字面量，Python 会自动把它们保存到相应对象的 doc 属性中。这对模块文件、函数 def、类和方法都适用。既然我们已经了解了更多类和方法的知识，示例 29-10（即 docstr.py）提供了一个快速但全面的示例，总结了文档字符串可

以出现在代码中的哪些位置。所有这些字符串都可以是三引号块，也可以是像这里这样更简单的一行字面量。

代码分析

Example 29-10. docstr.py

```
"I am: docstr.__doc__"                                #
def func(args):
    "I am: docstr.func.__doc__"                        #
    pass
class Klass:
    "I am: Klass.__doc__ or docstr.Klass.__doc__ or self.__doc__"
    #
    def method(self):
        "I am: Klass.method.__doc__ or self.method.__doc__"
        #
        print(self.__doc__)
        print(self.method.__doc__)
```

解释器如何处理这段代码：

1. 自动拾取首字符串：解释器把每个"结构体"（模块、函数、类、方法）体内第一条独立字符串字面量提取出来，存入对应对象的 doc 属性。模块级的第一条字符串成为 docstr.doc；func 体内的第一条成为 func.doc；类别内第一条成为 Klass.doc（同时被实例的 self.doc 继承到）。
2. 方法也能取文档：self.method.doc 是"限定方法名 → 继承查找命中 Klass.method → 函数对象的 doc"。注意 self.method 是绑定方法，绑定方法的 doc 转发给底层函数的 doc。
3. print(self.doc)：实例 self 的 doc 经继承查到类的 doc，于是打印出类别的第一行字符串。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

英文原文

The main advantage of documentation strings is that they stick around at runtime. Thus, if it's been coded as a docstring, you can qualify an object with its doc attribute to fetch its documentation (calling print on the result interprets line breaks if it's a multiline string):

```
$ python3 >> import docstr >> docstr.doc'I am: doc  
str.doc'>> docstr.func.doc'I am: docstr.func.doc'>> d  
ocstr.Klass.doc'I am: Klass.doc or docstr.Klass.doc or s  
elf.doc'>> docstr.Klass.method.doc'I am: Klass.metho  
d.doc or self.method.doc'>> x = docstr.Klass() >> x.  
method() I am: Klass.doc or docstr.Klass.doc or self.d  
oc I am: Klass.method.doc or self.method.doc
```

A discussion of the PyDoc tool, which knows how to format all these strings in reports and web pages, appears in Chapter 15. Here it is running its help function on our code:

```
>> help(docstr) Help on module docstr: NAME docstr
r - I am: docstr.doc CLASSES builtins.object Klass
class Klass(builtins.object) | I am: Klass.doc or docstr.
Klass.doc or self.doc | | Methods defined here: | | method(self) | I am: Klass.method.doc or self.method.doc
| | -----
----- | Data descriptors defined here: | | dict |
dictionary for instance variables | | weakref | list of weak
references to the object FUNCTIONS func(args) I am: docstr.func.doc
FILE ../LP6E/Chapter29/docstr.py
```

Documentation strings are available at runtime, but they are less flexible syntactically than # comments, which can appear anywhere in a program. Both forms are useful, and any program documentation is good (as long as it's accurate, of course!). As stated before, the Python "best practice" rule of thumb is to use docstrings for higher-level functional documentation and hash-mark comments for more fine-grained coding documentation.

中文翻译

文档字符串的主要优点是它们在运行时一直存在。因此，如果某内容被编码为文档字符串，你可以用 doc 属性限定一

个对象来获取它的文档（如果换行字符串是多行的，对它调用 `print` 会解释换行符）：

```
$ python3 >>> import docstr
>>> docstr.doc'I am: docstr.doc'>>> docstr.func.doc'I am: docstr.func.doc'>>> docstr.Klass.doc'I am: Klass.doc or docstr.Klass.doc or self.doc'>>> docstr.Klass.method.doc'I am: Klass.method.doc or self.method.doc'>>> x = docstr.Klass() >>> x.method() I am: Klass.doc or docstr.Klass.doc or self.doc I am: Klass.method.doc or self.method.doc
```

关于 PyDoc 工具的讨论（它知道如何在报告和网页中格式化所有这些字符串）出现在第 15 章。这里是在我们的代码上运行它的 `help` 函数：

```
>>> help(docstr) Help on module docstr: NAME docstr - I am: docstr.doc CLASSES builtins.object Klass class Klass(builtins.object) | I am: Klass.doc or docstr.Klass.doc or self.doc | | Methods defined here: | | method(self) | I am: Klass.method.doc or self.method.doc | | -----
-----| Data descriptors defined here: | | dict | dictionary for instance variables | | weakref | list of weak references to the object FUNCTIONS func(args) I am: docstr.func.doc FILE /.../LP6E/Chapter29/docstr.py
```

文档字符串在运行时可用，但语法上不如 `#` 注释灵活——注释可以出现在程序的任何地方。两种形式都很有用，任何程序文档都是好的（当然，只要它准确！）。如前所述，Python 的“最佳实践”经验法则是：用文档字符串做较高级的功能性文档，用井号注释（hash-mark comments）做更细粒度的编码文档。

深度理解

·核心概念：文档字符串是可被程序读取的"运行时文档"——doc 让文档从"人的注释"升级为"对象携带的数据"。模块、函数、类、方法四层皆可。

·底层实现：编译模块/类/函数时，编译器检测代码对象开头的 LOADCONST（字符串常量）——若第一条语句是字符串字面量，就把它绑定为 doc。类文档字符串还会"遗传"给实例（继承查找），所以 self.doc 能拿到类的文档。

·为什么这样设计：文档要"随着代码走"而且"机器可读"，这样 help()、IDE 悬浮提示、自动生成文档（Sphinx）都能直接在运行时/离线时读取，无需解析源代码文本。

·实际问题：团队协作时，函数/类第一行写文档字符串已成为事实标配；help(obj) 在交互摸索时是最快的"查资料"姿势。

·初学者误区：① 把 # 注释和 docstring 混为一谈——注释进不了 doc，docstring 必须"第一条语句"且是字符串字面量；② 以为"随便哪里的字符串都是 docstring"——只有结构体顶部第一条才算；③ 在该放 docstring 的位置放了一个赋值或 import——那就不是 docstring 了。

29.23 类与模块 (Classes Versus Modules)

英文原文

Finally, let's wrap up this chapter by briefly comparing the topics of this book's last two parts: modules and classes. Because they're both about namespaces, the distinction can be confusing. In short:- Modules — Implement data+logic packages. Are created with Python files or other-language extensions. Are used by being imported. Form the top level in Python program structure.- Classes — Implement new full-featured objects. Are created with class statements. Are used by being called. Always live within a module. Classes also support extra features that modules don't, such as operator overloading, multiple instance generation, and inheritance. Although both classes and modules are namespaces, you should be able to tell by now that they are very different things. We need to move ahead to see just how unique classes can be.

中文翻译

最后，让我们简要比较本书最近两部分讨论的主题：模块与类，为本章收尾。因为它们都与命名空间有关，区别可能令人困惑。简而言之：- 模块（Modules）——实现“数据+逻辑”包。用 Python 文件或其他语言扩展创建。通过被导入（import）来使用。构成 Python 程序结构的顶层。- 类（Classes）——实现功能完备的新对象。用 class 语句创建。通过被调用（call）来使用。总是生活在某个模块之内。类还支持模块没有的额外特性，例如运算符重载、多个实

例的生成和继承。虽然类和模块都是命名空间，但到目前为止你应该能看出，它们是非常不同的东西。我们需要继续前进，才能看到类究竟有多么独特。

深度理解

·核心概念：模块 = 静态的"包"，类 = 动态的"工厂"。模块是"容器 + 命名空间"，类在命名空间之上还叠加了"可调用 → 生成对象 + 继承 + 运算符重载"三大能力。

·底层实现：模块对象是 `types.ModuleType` 实例（字典 + 少量元数据，无 `class/bases` 意义上的继承，无 `call` 机制）；类对象是 `type` 实例（可调用、可继承、可被实例化）。从对象模型看二者就是不同物种。

·为什么这样设计：把"代码包"（模块）与"对象模板"（类）分开，让 Python 的代码组织（复用单元）与运行时对象模型（生产机制）各司其职——前者只管"文件、导入、命名空间隔离"，后者管"状态、行为、特化"。

·实际问题：组织大型项目时想清楚"这个量级该用模块还是类"——共享常量工具用模块（`from utils import config`），需要多份独立状态用类。

·初学者误区：① 以为"类就是一种模块"——类永远存在于模块中，但模块没有继承；② 以为"导入一个类就是用它了"——导入只是拿到类对象，还要调用（`x = MyClass(...)`）；③ 以为模块也可以被"继承"——不能，模块只能 `import`。

29.24 本章小结 (Chapter Summary)

英文原文

This chapter took us on a second, more in-depth tour of the OOP mechanisms of the Python language. We learned more about classes, methods, and inheritance, and we wrapped up the namespaces and scopes story in Python by extending it to cover its application to classes. Along the way, we encountered core OOP concepts such as abstract superclasses, class data attributes, namespace links, and manual calls to superclass methods and constructors. Now that we've explored all the basic mechanics of coding classes in Python, the next chapter turns to a specific facet of those mechanics: operator overloading. After that, we'll explore common design patterns, looking at some of the ways that classes are commonly used and combined to optimize code reuse. Before you read ahead, though, be sure to work through the usual chapter quiz to review what we've covered here.

中文翻译

本章带我们进行了关于 Python 语言 OOP 机制的第二轮、更深度的巡礼。我们学习了更多关于类、方法和继承的知识，并通过把命名空间与作用域故事扩展到类的应用，为它在 Python 中画上了句号。沿途，我们遇到了抽象超类、类数据属性、命名空间链接以及手动调用超类方法和构造函数等

核心 OOP 概念。既然我们已经探索了在 Python 中编码类的基本机制，下一章将转向这些机制的一个具体侧面：运算符重载。在那之后，我们将探索常见的设计模式，研究类经常被使用和组合以优化代码复用的一些方式。不过，在继续阅读之前，请务必完成通常的章节测验，复习我们在此覆盖的内容。

29.25 测试你的知识：测验 (Test Your Knowledge: Quiz)

英文原文

1. What is an abstract superclass? 2. What happens when a simple assignment statement appears at the top level of a class statement? 3. Why might a class need to manually call the init method in a superclass? 4. How can you augment, instead of completely replacing, an inherited method? 5. How does a class's local scope differ from that of a function?

中文翻译

1. 什么是抽象超类 (abstract superclass) ? 2. 当一条简单的赋值语句出现在 class 语句顶层时，会发生什么？ 3. 类为什么可能需要手动调用超类中的 init 方法？ 4. 你怎样才能增强 (augment) 而不是完全替换一个被继承的方法？ 5. 类的局部作用域与函数的局部作用域有何不同？

29.26 测试你的知识：答案 (Test Your Knowledge: Answers)

英文原文

1. An abstract superclass is a class that calls a method, but does not inherit or define it—it expects the method to be filled in by a subclass. This is often used as a way to generalize classes when behavior cannot be predicted until a more specific subclass is coded. OOP frameworks also use this as a way to dispatch to client-defined, customizable operations.

2. When a simple assignment statement ($X = Y$) appears at the top level of a class statement, it attaches a data attribute to the class (`Class.X`). Like all class attributes, this will be shared by all instances that do not have the same attribute. Methods are generally created instead by `def` statements nested in a class.

3. A class must manually call the `init` method in a superclass if it defines an `init` constructor of its own and still wants the superclass's construction code to run (and it often will). Python itself automatically runs just one constructor—the lowest one in the inheritance tree. Superclass constructors are often called through the class name, passing in the self instance manually: `Superclass.init(self, ...)`; they may also be called by `super().init(...)`, though we haven't yet studied this form in full.

4. To augment instead of completely replacing an inherited method

d, redefine it in a subclass, but call back to the superclass's version of the method manually from the new version of the method in the subclass. That is, pass the self instance to the superclass's version of the method manually: `Superclass.method(self, ...)`; or do so implicitly with `super().method(...)`. The prior answer is really just a special case of this one. 5. A class is a local scope and has access to enclosing local scopes, but it does not serve as an enclosing local scope to further nested code. Like modules, the class local scope morphs into an attribute namespace after the class statement is run.

中文翻译

1. 抽象超类是一个调用某方法、但并不继承或定义它、而是期望该方法由子类填空的类。当行为要等到编码出更具体的子类后才能预知时，这常被用作泛化（generalize）类的手段。OOP 框架也把它用作分派（dispatch）到客户端定义的、可定制操作的方式。2. 当一条简单赋值语句（ $X = Y$ ）出现在 class 语句顶层时，它会给类挂上一个数据属性（`Class.X`）。像所有类属性一样，它会被所有没有该同名属性的实例共享。方法则通常由嵌套在 class 中的 `def` 语句创建。3. 如果类定义了自己的 `init` 构造函数、又想超类的构造代码也运行（而且它常常想），就必须手动调用超类中的 `init` 方法。Python 本身只自动运行一个构造函数——继承树中最下方的那一个。超类构造函数通常通过类名调用，手动传入 `self` 实例：`Superclass.init(self, ...)`；也可以用 `super().init(...)` 调用，尽管我们还没有完整学习这种形式。4.

要增强而不是完全替换一个被继承的方法，就在子类中重新定义它，但从子类中该方法的新版本里手动回调超类版本。也就是说，手动把 self 实例传给超类版本的方法：Superclass.method(self, ...); 或者用 super().method(...) 隐式实现。上一题的答案其实只是本题的特例。5. 类是一个局部作用域，可以访问外层局部作用域，但它不充当进一步嵌套代码的包围局部作用域。像模块一样，类的局部作用域在 class 语句运行后就变形为属性命名空间。

本章总结

技术拓展 (Technical Expansion)

本章知识在真实项目中的应用场景：

- 框架/库的"钩子"设计：抽象超类 + 动态分派是 GUI 控件、Web 视图、ORM 基类、插件系统的通用骨架。理解了"超类定流程、子类填实现"，就能读懂 Django Model、Flask Blueprint、pytest 的 fixture 基类等大规模框架的骨架逻辑。
- 属性与命名空间排查：深入理解"赋值 vs 引用""实例字典 vs 类字典"后，遇到"改了不生效 / 值被共享 / 属性凭空出现"类 bug 时，可以按固定流程（先查 dict、再查 mro）快速定位。
- 工具与元编程：classtree 虽小，却演示了"用运行时特殊属性写通用工具"的思路——序列化（遍历 dict）、调试器（打印类树）、日志与监控（统一挂到基类）都是这一

思路的延续。

· OOP 规范落地：子类构造器显式调用 `Super.init(self, ...)`、抽象方法用 `@abstractmethod` 声明，这些是团队协作下保证初始化链和接口契约不被悄悄打破的实用规范。

与 Java/C++ 的核心差异（表格）：

特性	Python	Java / C++
类语句的性质	可执行代码，运行时创建类对象	编译期声明（C++ 还需分声明/定义）
静态数据成员	类体顶层赋值即可，无需关键字	需要显式 <code>static</code> 关键字（Java/C++）
方法须显式接收"当前对象"	必须写 <code>self</code> （显式）	<code>this</code> 隐式（Java/C++）
函数重载（同名不同签名）	不存在——后 <code>def</code> 覆盖前 <code>def</code>	支持签名级重载
抽象类	惯用 <code>abc.ABCMeta/@abstractmethod</code> ，也可手工 <code>raise NotImplementedError</code>	Java 用 <code>abstract</code> 关键字；C++ 用纯虚函数
继承查找	运行时动态查找（每指点号触发）	编译期静态解析（虚函数例外）
私有性	默认公开，用下划线约定 + 可选工具强制	编译期关键字强制（ <code>private/protected</code> ）
类的多重角色	类即对象（可存、可传、可动态生成）	类不是运行时对象（Java 的 <code>Class</code> 是元对象，不同概念）

Python 发展历史背景：

·本章的手工抽象超类例子（`assert/NotImplementedError`）反映了 Python 2 时代的惯例——`abc` 模块直到 Python 2.6 / 3.0 才进入标准库，此前"抽象基类"完全是社区约定。

· `super()` 的历史比较复杂：早期 `super` 只认"当前类 + 直接基类"，行为在多种继承下极易出错，因此本书明确建议

初学者先用显式写法——这与 Python 社区后来力推 `super()` (PEP 状态下的现代最佳实践) 形成有趣张力, 理解机制后再迁移即可。

- Python 3 中"所有类默认继承 `object`" (新式类统一) 让 MRO 与元类模型变得彻底一致, 本章出现的 `object` 自动入树就是这一设计的直观体现。

高级开发者需要掌握的相关知识:

- mro 与 C3 线性化 (第 31 章) ——多重继承时"到底先找谁"的精确规则。

- 描述符 (descriptors)、`getattr/setattr` (第 38 章) ——属性管理工具如何在两步协议上做拦截。

- `super()` 的协作式多重继承 (第 32 章) ——何时该放弃显式写法。

- 元类 (第 40 章) ——"类对象本身由谁创建"这一层的控制权。

- `property` (第 32 章) ——把方法伪装成属性的常见实践, 也是描述符的第一课。

学习建议 (Learning Advice)

- 重要程度: (5/5) ——本章是全书的"OOP 核心细节章"。前有第 27/28 章铺垫, 后有运算符重载、设计模式、MRO, 几乎后文所有类相关内容都以本章的概念为地基。

- 应该掌握到什么程度:

1. 背熟两条铁律：继承查找只发生在引用上、不发生在赋值上；`instance.method(a) ≡ Class.method(instance, a)`。
 2. 能徒手画出属性树：给定 class 定义，能说出 `x.attr` 的查找路径与最终命中对象。
 3. 能手写三类代码：子类构造器显式调用 `Super.init(self, ...)`；覆盖方法时用 `Super.method(self)` 做扩展；用 `raise NotImplementedError`（或 `@abstractmethod`）声明抽象方法。
 4. 能解释名字解析：裸名走 LEGB（且类不在 E 中），限定名走对象命名空间 + 继承查找；能说清 `manynames.py` 里五个 X 各自的归宿。
 5. 会用调试三件套：`obj.dict`、`obj.class`、`Class.bases / Class.mro`，以及 `dir()`。
- 建议在 REPL 中亲手复现 `SharedData`、`NextClass`、`specialize.py`、`classscope1-3`、`classtree` 的全部输出，再对照书内注释验收。
- 后续应该学习哪些相关内容：
1. 第 30 章：运算符重载（operator overloading）——`__init__` 只是这些魔法方法的第一个。
 2. 第 31 章：多态与继承算法正式化——“同名覆盖”与“接口约定”的深入解释（为什么没有函数重载）。
 3. 第 32 章：`super()`、`@staticmethod`/`@classmethod`、`property`、`slots`——本章所有“预告”的兑现现场。

4. 第 38/39/40 章（进阶）：属性管理工具、装饰器、元类——把本章的“规则 + 例外”放到最高视角俯视。

·若想立刻动手：把上一章 Manager 例子的构造链改用 `super()` 重写并对比行为；或为任意类写一个 `displaytree()` 通用工具。